

MetaML-Pro: Cross-Stage Design Flow Automation for Efficient Deep Learning Acceleration

ZHIQIANG QUE, Imperial College London, UK

JOSE G. F. COUTINHO, Imperial College London, UK

CE GUO, Imperial College London, UK

HONGXIANG FAN, Imperial College London, UK

WAYNE LUK, Imperial College London, UK

This paper presents a unified framework for codifying and automating optimization strategies to efficiently deploy deep neural networks (DNNs) on resource-constrained hardware, such as FPGAs, while maintaining high performance, accuracy, and resource efficiency. Deploying DNNs on such platforms involves addressing the significant challenge of balancing performance, resource usage (e.g., DSPs and LUTs), and inference accuracy, which often requires extensive manual effort and domain expertise. Our novel approach addresses two core issues: (i) encoding custom optimization strategies and (ii) enabling cross-stage optimization search. In particular, our proposed framework seamlessly integrates programmatic DNN optimization techniques with high-level synthesis (HLS)-based metaprogramming, leveraging advanced design space exploration (DSE) strategies like Bayesian optimization to automate both top-down and bottom-up design flows. Hence, we reduce the need for manual intervention and domain expertise. In addition, the framework introduces customizable optimization, transformation, and control blocks to enhance DNN accelerator performance and resource efficiency. We further formalize a cross-stage, constrained Bayesian optimization procedure that couples predicate-action bottom-up feedback (via BRANCH) with FORK/REDUCE order search, enabling automated selection, ordering, and tuning across software and HLS tasks. Experimental results demonstrate up to a 92% DSP and 89% LUT usage reduction for select networks, while preserving accuracy, along with a 15.6-fold reduction in optimization time compared to grid search. These results highlight the potential for automating the generation of resource-efficient DNN accelerator designs with minimal effort, resulting in large resource savings with bounded exploration cost.

CCS Concepts: • **Hardware** → **Electronic design automation**; **Emerging tools and methodologies**; **Hardware-software codesign**; • **Computer systems organization** → **Neural networks**; **Reconfigurable computing**.

Additional Key Words and Phrases: FPGAs, algorithm-hardware codesign, design automation, design space exploration, MetaML

1 Introduction

The field of deep learning has witnessed unprecedented growth in recent years, driven by the increasing demand for efficient and high-performance applications [23, 25, 44, 63]. Consequently, FPGA-based deep neural network (DNN) accelerator design and optimization have gained significant attention [38, 39, 42, 47]. The development of an efficient FPGA-based DNN design requires a diverse skill set that combines expertise in machine learning with

Authors' addresses: Zhiqiang Que, z.que@imperial.ac.uk, Imperial College London, UK; Jose G. F. Coutinho, gabriel.figueiredo@imperial.ac.uk, Imperial College London, UK; Ce Guo, Imperial College London, UK; Hongxiang Fan, Imperial College London, UK; Wayne Luk, w.luk@imperial.ac.uk, Imperial College London, UK.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1936-7414/2026/2-ART

<https://doi.org/10.1145/3795794>

low-level knowledge of the target hardware architecture [40, 55]. Optimizing these DNN designs is a complex process, as it involves balancing competing objectives [34]. On one hand, high accuracy during inference is crucial from an application perspective. On the other hand, the design must be optimized for the underlying hardware architecture, meeting power, latency, and throughput requirements while fitting into FPGA devices [35, 43]. Achieving an optimal balance between these conflicting objectives requires careful consideration and effective optimization strategies [34].

Existing optimization techniques for DNNs and hardware have a limitation in that they are often handled in separate stages, with information exchanged only in a top-down manner from the application to hardware. Bottom-up information flow, where hardware design insights inform DNN model optimization, typically requires manual intervention. However, this information exchange across multiple stages can significantly enhance the optimization process, as application and hardware optimizations can interact and affect each other. For instance, modifying the DNN model architecture could impact the FPGA resource utilization, and optimizing the FPGA design could influence the DNN model's accuracy [35, 62]. In addition, there has been limited focus on combining optimization strategies targeting different abstraction levels, such as algorithmic level neural networks and High Level Synthesis (HLS) C++, hindering the selection of the most effective combination of optimization techniques for a given problem. In summary, only a limited number of studies have thoroughly explored the vast configuration space that emerges when combining DNN optimization techniques across both software and hardware domains.

In this paper, we address these aforementioned technical challenges as follows:

- **C1. Custom Co-Optimization Strategies.** First, this work employs a unique co-optimization approach that refines both the DNN software model and the hardware architecture. By encoding software and hardware optimizations independently and composing them as needed, this approach enables modular reuse and adaptation of optimizations across different benchmarks and platforms. Through programmatic manipulation, it ensures tight coordination between DNN model and hardware-level optimizations, significantly expanding the design space and enhancing exploration capabilities.
- **C2. Cross-Stage Optimization Search.** Second, developing efficient DNN accelerators requires a coherent strategy that automates the selection, combination, and tuning of optimization techniques across multiple abstraction levels [2, 30, 66, 68]. It is challenging to identify the most effective combination, order and tuning of optimization techniques to ensure optimal outcomes [65, 69]. This work addresses these challenges by employing a cross-stage optimization search approach that coordinates the interaction between different stages of the design flow. By leveraging a combination of top-down and bottom-up optimization strategies, the framework dynamically adapts to the specific requirements of the hardware and the application, guiding the optimization process iteratively and identifying the best optimization techniques and their sequences.

Our proposed co-optimization framework, as illustrated in Fig. 1, operates across various computational spaces, such as software and High-Level Synthesis (HLS), and can target diverse FPGA platforms (see **C1**). Each computational space represents an autonomous, custom optimisation task (e.g., OPT-1 to OPT-5, and potentially more), governed by tunable parameters. Moreover, this work utilizes a Bayesian [26, 34, 57] guided nested-loop optimization structure involving both local and global phases, combining very distinct optimization tasks and incorporating feedback from the latter stages of the design flow to refine the process iteratively (see **C2**). Finally, while we instantiate Bayesian optimization for the outer DSE, the controller is algorithm-agnostic and can host evolutionary or RL-based search as drop-in alternatives.

We make the following contributions in this paper:

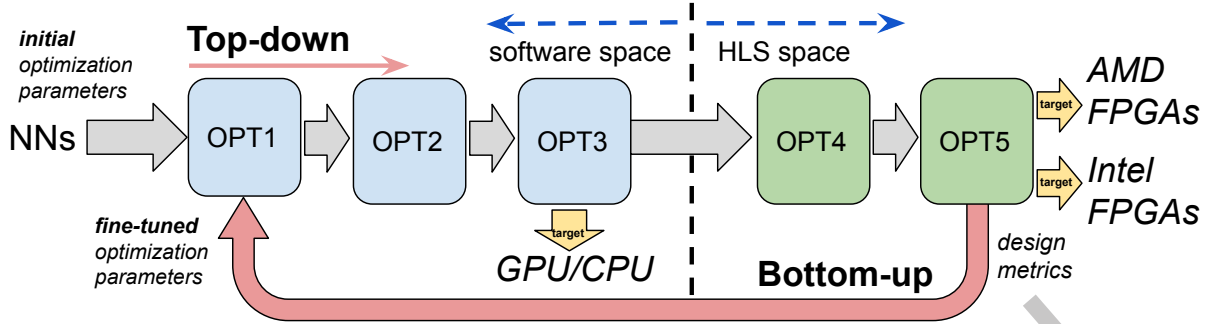


Fig. 1. The proposed approach.

- A novel co-optimization framework that enables modular and adaptive hardware-software exploration for FPGA-based DNN accelerators. It independently encodes and combines optimizations for efficient design refinement.
- A cross-stage optimization algorithm. We formalize a nested-loop, constraint-aware Bayesian optimization procedure over software- and HLS-level optimization tasks. The algorithm uses predicate-action bottom-up feedback (via BRANCH) to adapt accuracy/utilization tolerances on constraint violations, and FORK/REDUCE to search task permutations and aggregate by Pareto dominance. This automates selection, ordering, and tuning across abstraction levels.
- A library of reusable optimization, transformation, and control tasks designed to be customizable and flexible, and that can be easily integrated into our co-optimization framework. This library includes optimization tasks such as pruning, quantization, and scaling to enhance DNN accelerator performance and resource efficiency. Moreover, we leverage metaprogramming to programmatically analyze and modify HLS C++ designs, enhancing design flexibility, performance tuning, and resource efficiency. Some of the tasks in our library are specific to certain applications and target technologies, while others remain agnostic, providing versatility and adaptability within the framework.
- A comprehensive evaluation of the proposed framework using multiple benchmarks and different optimization strategies. This evaluation provides insights into the effectiveness of the framework and its optimization modules under different scenarios (Section 5).

To the best of our knowledge, this is the first automated framework that combines reusable optimization components with adaptive feedback and global exploration to jointly balance resource efficiency and accuracy in deep learning designs.

Relationship to Prior Publications. This paper expands on our previous studies [33, 36, 46] which primarily focus on local optimizations. In [36], we propose MetaML, an optimization framework, that customizes design flow for DNNs with local optimizations. In [33], we introduce control capabilities within the optimization framework, while [36] focuses on Quantization Heuristic Search (QHS) optimization task for compressing DNN accelerators. However, these prior studies are limited in their ability to perform global optimization due to their focus on isolated optimization tasks. The inability to account for interdependencies between optimization tasks across stages mean that the overall design might not have achieved the optimal balance between accuracy, performance, and hardware resource. This work addresses these limitations by incorporating bottom-up feedback and leveraging Bayesian optimization to enable global optimization across the entire design flow. By adjusting optimization tasks at each stage based on performance feedback, our framework ensures that each optimization choice contributes to a more cohesive and globally optimal solution. This extended approach maintains high accuracy while improving resource efficiency, specifically in resource-constrained environments.

2 Related Work

The field of FPGA-based DNN acceleration has rapidly expanded, leading to the development of numerous optimization techniques and tools. Several co-optimization techniques have been proposed that optimize both algorithm and hardware stages for DNNs on FPGAs, as discussed in various papers [8, 14–18, 20, 66, 68] and in hardware-aware neural architecture search studies like [2, 10, 30]. While hardware-aware neural architecture search (HW-NAS) approaches typically involve a holistic approach where the search algorithm must consider both software and hardware aspects together, often requiring a broader range of expertise within both NAS and hardware, our work modularizes the optimization process, allowing domain experts to focus on their areas of expertise. In addition, this modular design allows for individual optimization tasks to be reused independently in different contexts or different models, while the holistic nature and tight coupling of HW-NAS makes it challenging to reuse parts of the optimization process independently. This work can independently update and refine specific modules without affecting the entire system, resulting in better reusability and adaptability.

Other approaches offer end-to-end software frameworks, such as Xilinx’s Vitis AI [21] and Intel’s OpenVINO [1], which optimize DNNs with pre-built optimizations for deployment on specific target technologies. However, they are not designed for easy addition of new optimization strategies or to search for the most effective combined strategies.

Furthermore, some frameworks allow developers to describe and customize DNN optimization strategies, but their scope is limited. For instance, ScaleHLS [67] and SOTA-OPT [3] focus on HLS-based optimization strategies and hardware designs, but their optimization scope is restricted to MLIR. TVM [5] is a general-purpose DNN compilation framework that offers performance portability across different types of devices, but optimizations occur solely at the graph (IR) level. da4ml [54] is an open-source library that compiles machine-learning models into hardware designs. It targets fully pipelined implementations with an initiation interval (II) of 1, applies distributed arithmetic (DA) optimizations, and can emit synthesizable Verilog and VHDL for multiple FPGA back-ends.

Moreover, frameworks, such as FINN [59], HLS4ML [50], and fpgaConvNet [61], provide optimized hardware building blocks for FPGA-based DNN accelerators, and allow optimization strategies to be codified using these blocks. However, they do not support automated bottom-up optimization flows, where hardware stage information guides the optimization of the application (DNN) stage.

To overcome the limitations of existing optimization techniques for DNNs and hardware, our framework provides fully automated high-level optimization flows backed by reusable target-specific and target-agnostic building blocks. More specifically, our approach enables automated top-down and bottom-up flows, allowing for the creation of customized cross-stage optimization strategies for various DNN designs. Moreover, our approach integrates new and existing optimization techniques targeting various levels of abstraction, such as neural networks through graph optimizations and HLS C++ through source-to-source optimizations [60].

3 Our Approach

3.1 Overview

In this paper, we introduce a novel framework for creating customizable design-flows that optimize FPGA-based DNNs, with a focus on automating both top-down and bottom-up flows between application and hardware optimization stages.

A design-flow represents a sequence of tasks that convert a high-level specification into a final hardware design. A design-flow is typically implemented as a multi-stage pipeline, with each stage operating on a specific model abstraction. As the pipeline progresses, the model abstraction is gradually reduced and optimized, taking into account the features of the target device. For example, a typical design-flow targeting FPGAs is shown in Fig. 2. It begins with defining specifications (SPEC) and testing the design in software (SW). If High-Level Synthesis

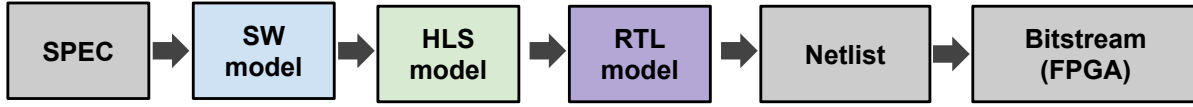


Fig. 2. A typical FPGA design flow. This FPGA design flow begins by defining the specifications (SPEC), followed by implementing and testing the design in software (SW). If High-Level Synthesis (HLS) is used, high-level code (e.g., C/C++) is converted into hardware description language (HDL). The design then moves to the Register Transfer Level (RTL) stage, where it is described in HDL (e.g., Verilog/VHDL) and synthesized into a netlist of logic gates. Finally, the netlist is used to generate a bitstream, which is loaded onto the FPGA to configure the hardware. Each stage progressively refines the design from concept to implementation.

(HLS) is used, a high-level DNN model, such as one described in TensorFlow, can be translated into a C++ HLS model using tools like HLS4ML. This model is then converted into an RTL description (e.g., Verilog/VHDL) using tools like Vivado HLS. The RTL model is further synthesized into a netlist, which is optimized for the FPGA's architecture, and finally translated into a bitstream that configures the FPGA hardware.

Feedback information can be utilized to enable lower-stage information to refine higher-stage optimizations, employing more accurate metrics that were previously unavailable. We have developed a framework that allows users to codify fully automated optimizing design-flows with the following requirements:

- **Customizable:** Refers to the ability for users to customize, extend, or modify the design-flow to meet specific needs and support experimentation. Users can select a set of building tasks that implement optimization, transformation, or control tasks, combine them in a specific order, and fine-tune their parameters to generate a specific optimization flow. Additionally, they can create their own tasks and integrate them into their design-flow (see C1);
- **Multi-level:** Refers to the scope of optimizations and the ability to target multiple levels of abstraction, typically associated with different stages of a design-flow. For instance, optimizations operating at the Neural Network level are performed at the graph-level, while optimizations at the HLS C++ level are performed using source-level transformations (see C1);
- **Cross-stage:** Refers to the capability of a design-flow to support top-down and bottom-up flows between optimization stages, such as software and hardware, respectively (see C2).

This work focuses on custom optimizations in the SW and HLS stages while using default configuration for the remaining stages, however, the same concepts can be extended to the other stages, which is left for future work.

3.2 Architecture

Fig. 3 illustrates the key architectural components of a programmatically generated design flow using our approach, which comprises two main components: the pipe task and the meta-model.

The pipe task is the fundamental building block of the design flow and is responsible for implementing specific a task, such as optimization or control. By connecting pipe tasks, we create a complete design flow. The design flow's internal architecture is represented by a cyclic directed graph, where nodes represent tasks, and edges represent **paths** between tasks. A path represents a dependency between two tasks, such that a task can only be executed if its dependencies have finished executing.

The meta-model, on the other hand, is responsible for storing all of the design flow's states, including the configuration parameters of each pipe task and their respective outputs. Instead of communicating directly with each other, pipe tasks share information through the meta-model, which serves as a shared space. The meta-model has three sections, as illustrated in Fig. 3(b): configuration, log and model space.

The configuration section (**CFG**) is a key-value store that keeps the parameters of all pipe tasks in the design flow. These parameters can configure all pipe tasks of the same type or specific instances, and they can be

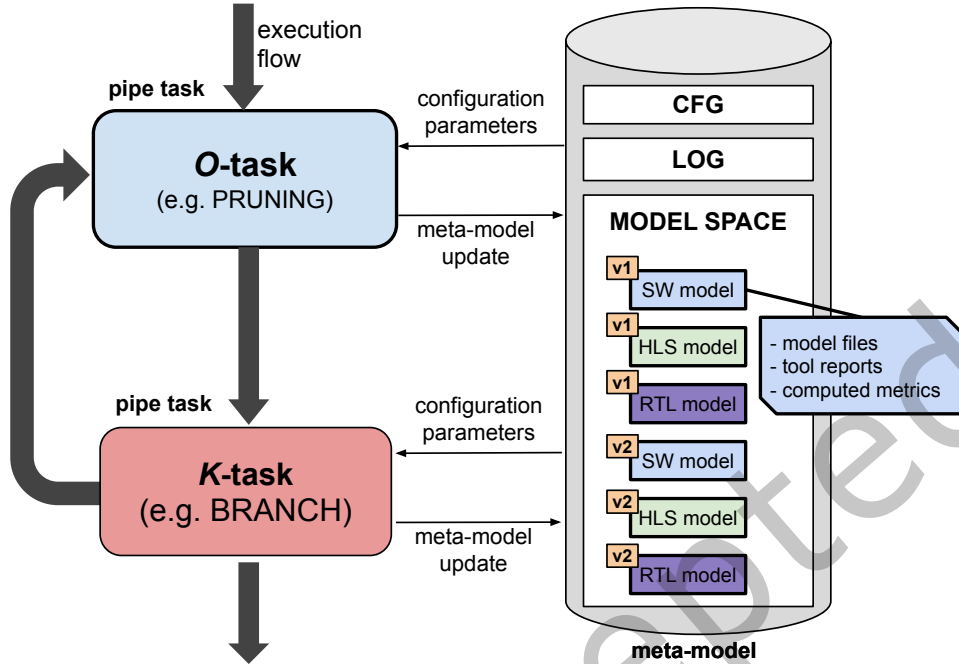


Fig. 3. A connection between a *O*-task and a *K*-task. A pipe task has a uniform interface allowing any two pipe tasks to be connected (although there may be constraints about how many connections a task can handle). A *O*-task typically enhances DNN models based on specific objectives and constraints. A *K*-task on the other hand, manages the control flow. Each connection defines a unidirectional stream between a source task and a target task.

supplied by the user or automatically modified by the pipe tasks as part of an automated tuning process. The log section (**LOG**) stores the runtime execution trace of the design flow. This section can be useful for debugging and understanding the execution flow of the design.

Finally, the **model space** stores the models generated during the execution of a design flow. Each stored model is versioned, and models derived by different stages can coexist in the same model space. In the example shown in the figure, we have stored six models covering three abstraction levels: DNN, HLS C++ , and RTL. Each model in the model space encapsulates all its supporting files, tool reports, and computed metrics. Moreover, they can be accessed and manipulated by any pipe task in the design flow, enabling cross-stage optimizations.

3.3 Reusable Pipe Tasks

Table 1 presents a list of pipe tasks that have been implemented in our approach, along with their roles, multiplicity, and parameters. The multiplicity denotes the number of input and output channels that a task can handle. We classify pipe tasks into three types based on their roles:

- **K-task**: These are generic tasks that control top-down and bottom-up flows. Examples include: **BRANCH**, which selects an output path at runtime based on a boolean condition; **JOIN**, which merges multiple paths into one; **FORK**, which allows multiple concurrent strategy paths; **REDUCE**, which consolidates the results of multiple strategy paths into one; and **STOP**, which terminates the design flow;
- **O-task**: These are self-contained optimizing tasks that enhance deep learning models based on specific objectives and constraints. Our current pipe task repository includes **PRUNING** and **SCALING**, which are

Table 1. A list of implemented pipe tasks.

Type	Role	Multiplicity	Parameters
JOIN	K	many-to-1	-
BRANCH	K	1-to-2	fn: meta-model $\rightarrow$ bool
FORK	K	1-to-many	-
REDUCE	K	many-to-1	fn: [meta-model] $\rightarrow$ meta-model
STOP	K	1-to-0	fn: meta-model $\rightarrow$ output
HLS4ML	λ	1-to-1	default_precision IOType FPGA_part_number clock_period test_dataset
VIVADO-HLS	λ	1-to-1	project_dir
INTEL-oneAPI	λ	1-to-1	project_dir
KERAS-MODEL-GEN	λ	0-to-1	train_en train_test_dataset train_epochs
PRUNING	O	1-to-1	tolerate_acc_loss (α_p) pruning_rate_thresh (β_p) train_test_dataset train_epochs
SCALING	O	1-to-1	default_scale_factor tolerate_acc_loss (α_s) scale_auto max_trials_num train_test_dataset train_epochs
QUANTIZATION	O	1-to-1	tolerate_acc_loss (α_q) train_test_dataset

implemented using the Keras API (version 2.9.0), and QUANTIZATION, which is performed using C++ source-to-source transformations via the Artisan framework [60];

- **λ -task:** These tasks perform functional transformations on the model space, such as compilation and synthesis. Examples include HLS4ML (version 0.6.0), which translates a DNN model into an HLS C++ model, and Vivado HLS (version 20.1), which translates an HLS C++ model into an RTL model.

It is worth noting that our framework is customizable, allowing users to create and integrate their own pipe tasks tailored to their specific needs.

```

1 # The pruning strategy architecture - Fig. 15(a)
2 from MetaML import *
3 # design-flow architecture
4 with Dataflow() as df:
5     join = Join() << KerasModelGen()
6     branch = Branch('B') << (VivadoHLS() <<
7         (HLSML() << (Pruning() << join)))
8
9     branch >> [join, Stop()]
10 # design-flow configuration
11 cfg = {
12     'KerasModelGen::train_en': False,
13     'Pruning::tolerate_accuracy_loss': 0.02,
14     'Pruning::pruning_rate_threshold': 0.02,
15     'B@fn': lambda metamodel: ...,
16     'HLS4ML::default_precision': 'ap_fixed<18,8>',
17     'HLS4ML::IOType': 'io_parallel',
18     'HLS4ML::FPGA_part_number': 'xc7z020clg400-1',
19     'HLS4ML::clock_period': 10,
20     'train_test_dataset': '/mypath/target_dataset',
21     'train_epochs': 15,
22     'Stop::fn': lambda metamodel: ...
23 }
24 # run design-flow
25 optimised_model = df.run(cfg)

```

Listing 1. The implementation of the pruning strategy using our framework.

3.4 Building a Strategy

Listing 1 provides the Python code for implementing the pruning strategy depicted in Fig.9(a) and discussed in Section 5.2. The code consists of three sections: **(i)** lines 4–9 builds the design-flow architecture graph by connecting the corresponding tasks. The `>>` and `<<` operators are utilized to define the connections between task instances, and the default name of each task instance serves as its identifier, although it can be overridden by providing it as an argument (line 6). The end result is a cyclic-directed graph. **(ii)** lines 11–23 define the configuration object, which holds all the parameters for configuring the tasks. Parameters referred to as `TaskType::parameter` are passed to all tasks of that task type (lines 16–19), whereas parameter names in the format of `InstanceName@parameter` are passed to a specific instance (line 15). All other parameters are global (lines 20–21) and accessible to any task. Finally, **(iii)** line 25 executes the design-flow by passing the configuration object.

The execution of the design-flow starts with graph validation, which ensures that there is at least one source task and no multiplicity constraints are violated. Tasks are executed by an internal scheduler using a thread pool, which assigns jobs to available workers or stalls until a worker becomes available. After validating the architecture graph, an empty meta-model is generated using the supplied configuration, and the scheduler runs the source task. When a task completes execution, it submits a job to execute the next set of tasks in the flow. The STOP task terminates execution after all submitted jobs are completed. Modifying the design-flow architecture and updating its configuration parameters can lead to new optimization strategies, which is discussed next.

Although this implementation may resemble a Python loop that sequentially invokes a series of functions, the Dataflow layer supports a fundamentally different architectural model to codify a design-flow. Rather than encoding a fixed linear script, it realizes a blackboard architecture [4], where each $O/K/\lambda$ -task is an independent

and potentially heterogeneous component, operating at the software, HLS, or hardware-report level, that does not invoke other tasks directly. All interaction instead occurs through the shared meta-model, which acts as the blackboard and stores model versions across abstraction levels, configuration parameters, tool reports, metrics, and execution logs in a unified structure. A controller uses this evolving state to determine which task should run next, enabling behaviors such as parallel exploration (FORK), conditional re-routing based on hardware constraints (BRANCH), merging of alternative paths (REDUCE), and revisiting earlier stages when necessary. This design allows heterogeneous components to be composed without predefined interfaces and provides the minimal structure required for cross-stage, feedback-driven optimization, which cannot be captured by a simple loop-based implementation.

4 Implementation

4.1 Co-Optimization Workflow

As illustrated in Fig. 4, our co-optimization approach allows the encoding of optimizations across different models of computation - from software to hardware (see **C1** in Section 1). At the top level, a DSE optimization program is employed (see **C2**), executing a comprehensive exploration strategy and overseeing local optimizations at two (or more) distinct levels of abstraction. At the software DNN level, optimizations are executed on the architecture using tools like Tensorflow or PyTorch. At the HLS DNN level, optimizations are conducted by modifying the HLS code automatically using metaprogramming. Optimizing DNN models across multiple abstraction levels, from software to hardware, leverages complementary strategies that address different bottlenecks within the system. Software optimizations prioritize algorithmic efficiency and reducing computational complexity, while hardware optimizations target improvements in throughput, latency, resource utilization and energy efficiency.

Our co-optimization workflow begins with the Exploration space. Here, a DSE program utilizes a DNN model, training data, and configuration parameters to conduct the exploration workflow. This process yields a variety of hardware designs with trade-offs in metrics such as performance and accuracy. A controller process orchestrates this exploration by managing communication between the DSE program and local optimization spaces. Prior to exploration, the DSE program sets up and deploys optimization programs within these spaces. Fig. 4 depicts two such spaces: one for software (SW) utilizing TensorFlow and another for HLS employing Vivado HLS. Each space optimizes the model according to selected strategies, such as pruning or quantization.

After system setup, the exploration loop commences. In each iteration: ❶ The DSE program dispatches the DNN model and a specific parameter set to the software optimization space, where tasks like scaling and pruning are executed. ❷ Upon optimization completion, the refined model is returned to the controller along with its associated accuracy. ❸ Subsequently, the model is forwarded to the HLS space for additional optimization. A metaprogram adjusts the source code based on recommended optimizations (e.g., quantization) and parameter values from the controller. After synthesis, metrics such as power, area, and accuracy are obtained and ❹ fed back to the Exploration space. The DSE program evaluates designs using these metrics, retains promising candidates, and suggests new parameters for the next iteration.

This separation of concerns into distinct optimization spaces—software, HLS, and DSE—allows for the integration of diverse optimization strategies and techniques. It enables the introduction of faster or more effective exploration methods without altering existing optimizations. In addition, it combines exploration across software and hardware domains to identify effective techniques for optimizing performance. Moreover, it facilitates specialization of new hardware optimizations specific to certain FPGA HLS tools by allowing HLS code to be customized through metaprogramming.

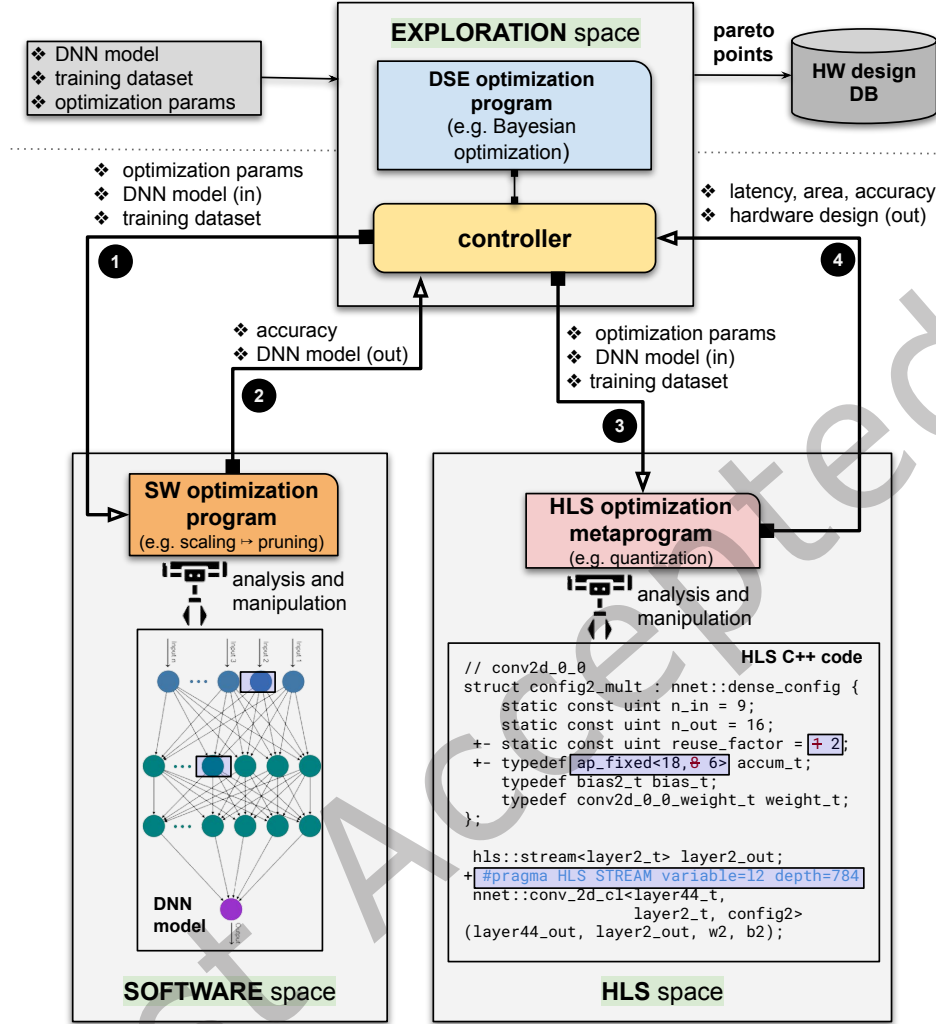


Fig. 4. This figure illustrates the implementation of our co-optimization framework, featuring an organized system of *optimization* spaces, each autonomously running Python programs within dedicated environments, overseen by an *exploration* space executing a general optimization strategy. The diagram depicts the orchestration of local optimization spaces, such as software and hardware, through a controller process.

4.2 Auto-Pruning with Binary Search

Pruning is a technique that improves the performance and efficiency of neural networks by removing insignificant weights. The pruning strategy uses a PRUNING *O*-task, which gradually zeroes out weights during training to create a more compact and efficient network while maintaining accuracy. This optimization task supports auto-pruning, which automatically determines the highest pruning rate while maintaining a given level of accuracy loss. The PRUNING *O*-task is illustrated in Fig. 9(a). Formally, the objective of this *O*-task is defined as:

$$\begin{aligned}
 &\text{maximum} && \text{Pruning_rate} \\
 &\text{subject to} && \text{Accuracy_loss}(\text{Pruning_rate}) \leq \alpha_p
 \end{aligned} \tag{1}$$

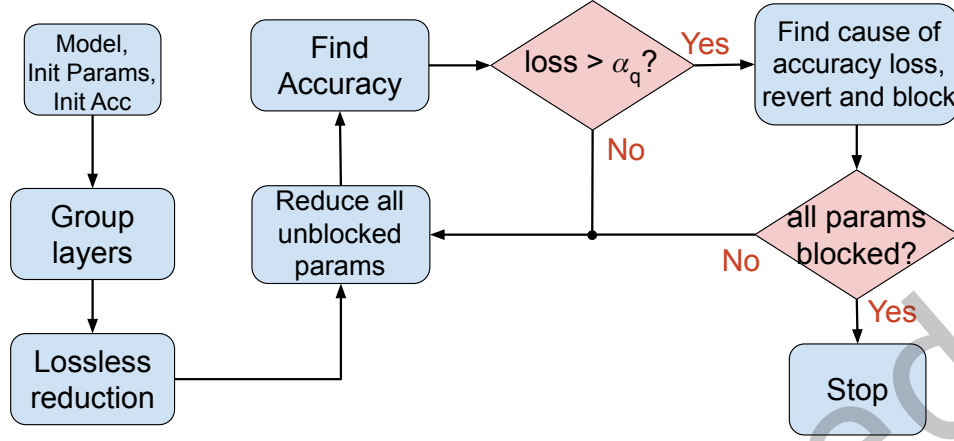


Fig. 5. The proposed QHS algorithm

Starting at 0% pruning rate, the auto-pruning algorithm obtains initial accuracy Acc_{p0} at step 1 (s1). It then uses a binary search approach, increasing or decreasing the pruning rate based on whether the accuracy loss is within a user-defined tolerance ($\leq \alpha_p$) as shown in Table 1. The algorithm terminates when the rate difference is below a threshold (β_p). The number of steps is determined by $1 + \log_2(1/\beta_p)$.

4.3 Auto-Scaling

To accommodate large DNN designs on FPGAs, we adopt the SCALING *O*-task that automatically reduces the layer size while monitoring the result accuracy loss, denoted by α_s . The objective of this *O*-task is formally defined as:

$$\begin{aligned} &\text{minimum} \quad \text{Scaling_factor} \\ &\text{subject to} \quad \text{Accuracy_loss}(\text{Scaling_factor}) \leq \alpha_s \end{aligned} \quad (2)$$

The auto-scaling process begins with a scaling factor of 1 (i.e., no scaling), getting initial accuracy Acc_{s0} at step 1 (s1). The algorithm then gradually reduces the scaling factor, continuing the search until the accuracy loss exceeds the threshold α_s . If necessary, α_s can be adjusted to achieve further size reduction with minimal impact on accuracy.

4.4 Auto-Quantization with Heuristic Search

In our framework, we introduce quantization task based on the Quantization Heuristic Search (QHS) algorithm [46], a post-quantization method. It automatically determines the lowest bitwidth for each layer while maintaining a given level of accuracy loss. Formally, the objective of this task is defined as:

$$\begin{aligned} &\text{maximum} \quad \text{Bit_width_reduction} \\ &\text{subject to} \quad \text{Accuracy_loss}(\text{Bit_width_reduction}) \leq \alpha_q \end{aligned} \quad (3)$$

It performs mixed-precision quantization for DNNs, allowing separate precisions for the weights, biases and results of layers of a model, taking the advantages of the customizability of FPGAs. While this customizability creates a massive search space of quantization configurations, dependencies within the network can be used to reduce the degrees of freedom of the design space.

Fig. 5 shows the steps of the proposed QHS algorithm. The optimization task begins by accepting a model with a maximum tolerable accuracy loss, denoted as α_q . Initially, virtual layers are created, and lossless reduction is performed by minimizing the integer bits representing model parameters using the base-2 logarithm of the largest

weight and bias in each layer, plus one bit for the sign to avoid saturation. Subsequently, all parameters (weights, biases and results) are initially assumed to be further reducible. The total bit-widths of these parameters are then reduced by one bit if they are considered reducible. This quantization is applied to the C++ kernel, and the accuracy is evaluated through a runtime simulation. If the accuracy loss remains within the user-specified tolerance, α_q , this step is repeated. If the accuracy loss exceeds the tolerance, the algorithm identifies and blocks non-reducible precisions. It determines which precisions are most sensitive by testing the reduction impact on accuracy. If a bit-width reduction breaks the accuracy constraint, that precision is marked non-reducible. The process repeats until no further reductions are possible within the accuracy constraints. Support for Quantization-aware training, such as HGQ [53], is left to future work.

4.5 Metaprogramming for HLS Design Optimization

A novel aspect of our co-optimization approach is the integration of metaprogramming into hardware design optimization, which enables the implementation of the Quantization Heuristic Search (QHS) algorithm described in Section 4.4. In this section, we describe the key motivation and benefits of metaprogramming, and how we integrate it in our co-optimization framework.

Beyond template-based and code generator approaches. Template-based frameworks and code generator modifications (for instance, extending HLS4ML) are effective when optimization choices can be captured as a fixed set of global parameters. For example, templates can expose settings such as default precision, loop unroll factors, or hardware interface options. Similarly, modifying a code generator allows these parameters to be realized directly in generated code. However, both approaches share two fundamental limitations:

- **Restricted scope of changes:** Templates and generators can only express transformations that were foreseen and encoded by their designers. Adding new optimisation dimensions requires extending the generator itself, which is costly and often tool-specific.
- **Limited applicability across sources:** In practice, real workflows mix generated code (e.g., HLS4ML) with user-written kernels, vendor libraries, and legacy C++. Template extensions or generator patches cannot easily propagate optimisations across such heterogeneous sources.

Benefits of metaprogramming. Our metaprogramming approach addresses these limitations by operating directly on the C++ Abstract Syntax Tree (AST). This enables:

- Local, structural optimizations such as loop-specific pragma placement, which templates cannot expose without proliferating knobs.
- Cross-kernel consistency edits, for example rewriting all types in a conv $\rightarrow$ pool $\rightarrow$ batch norm cascade to ensure coherent quantisation without re-generation.
- Device-aware refactorings guided by design-space exploration, where metaprograms inject vendor-specific constructs based on feedback.

Quantization illustrates this distinction clearly. A template or generator can reduce precision globally (e.g., set all layers to 8-bit), but cannot ensure type consistency across intermediate streams or flexibly adapt precision to device constraints without modifying the generator itself. Our metaprogram, by contrast, queries the AST to identify cascaded kernels, rewrites typedefs and template arguments in one pass, and produces a coherent low-precision dataflow candidate automatically.

In summary, templates and generators are well-suited for global configuration, while metaprogramming is essential when optimisation requires local, structural, or device-specific edits. This generality allows the same refactoring (e.g., quantisation) to be applied not only to HLS4ML output, but equally to handwritten or third-party C++ code, demonstrating the broader utility of our approach.

Example of an HLS metaprogram. Fig. 6(b) illustrates an HLS metaprogram that dynamically modifies a given C++ source file using Python. The process begins with the construction of an Abstract Syntax Tree (AST)

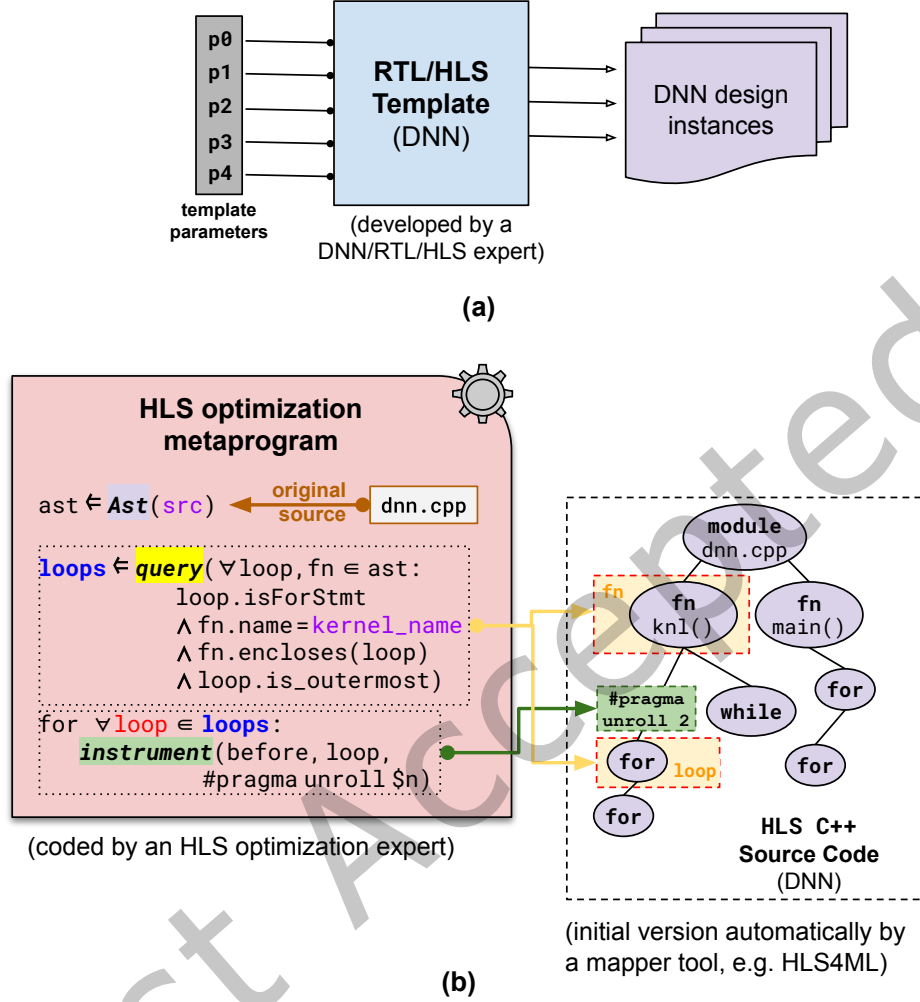


Fig. 6. (a) Current methods for deriving hardware designs rely on templates developed by experts. However, these templates are constrained in generating a finite number of design variants from predetermined parameter values and demand expertise for accommodating new DNN architectures. (b) An HLS metaprogram in pseudocode illustrating dynamic C++ code manipulation through Abstract Syntax Tree construction to derive an optimized design variant.

representation of the C++ code. The AST provides a structured representation of the source code, allowing precise modifications for any of its constructs. The metaprogram then performs code querying and analysis to identify outermost loop structures within kernel functions. These loops are critical for performance optimization, as their execution directly impacts resource utilization and latency.

Once the relevant loops are identified, the metaprogram instruments the code by inserting pragma directives such as `#pragma unroll $n`. These directives control loop unrolling dynamically, enabling optimized parallel execution of computations. By automating this process, the metaprogram allows rapid exploration of different optimization strategies without requiring manual code modifications. This automation significantly improves

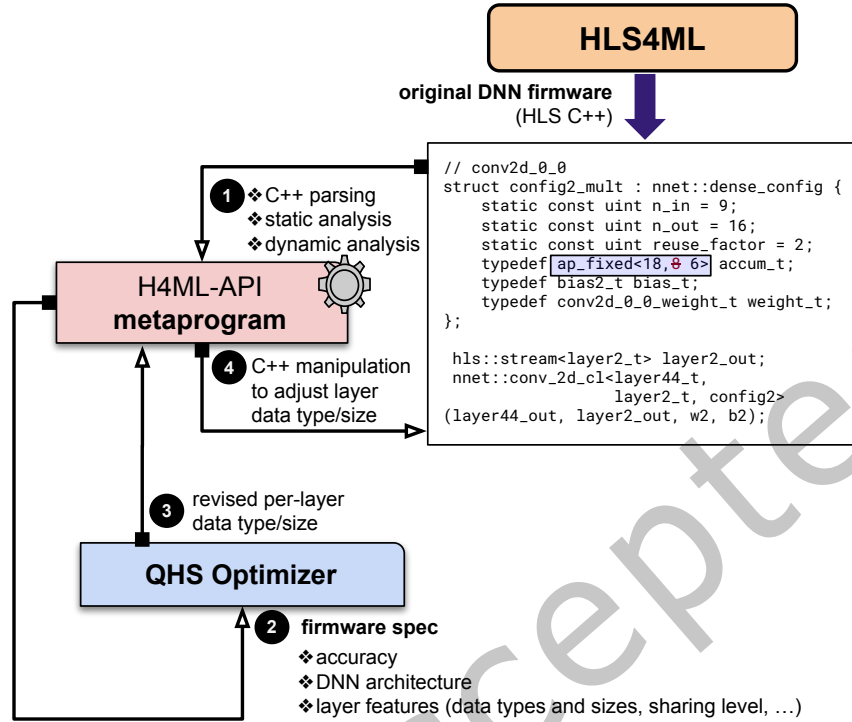


Fig. 7. Integration of the Quantization Heuristic Search (QHS) algorithm with HLS4ML and metaprogramming. The metaprogram dynamically adjusts data types and bit-widths at the HLS C++ level, optimizing computational efficiency while maintaining model accuracy.

both efficiency and design flexibility, making it easier to adapt to different DNN architectures and FPGA platforms. A more detailed explanation about querying and instrumentation capabilities can be found in [60].

Application in quantization optimization. We integrate Quantization Heuristic Search (QHS), detailed in Section 4.4, as an optimization task within our framework using metaprogramming to refine fixed-point numerical representations directly at the HLS C++ level. As illustrated in Fig. 7, we develop a metaprogram called H4ML-API, which programmatically analyzes and modifies HLS4ML-generated code to automate the quantization process.

The workflow proceeds as follows. First, the HLS4ML tool generates an initial, unoptimized firmware from a DNN model. Then, ① the H4ML-API metaprogram inspects the source structure and executes the firmware to extract DNN architecture details, layer characteristics, and accuracy metrics. This information is ② passed to the QHS process, which identifies opportunities for bit-width reduction while maintaining model accuracy, dynamically adjusting bit-widths based on layer-specific requirements. The refined bit-width settings are then ③ transferred back to the H4ML-API, which ④ modifies the HLS C++ code accordingly. This iterative process continues from step ① while the accuracy loss remains within the α_q threshold.

A key feature of this approach is the creation of virtual layers to optimize bit-width allocation. Rather than treating each operation separately, the metaprogram groups cascaded operations into logical units, enabling more effective quantization. For instance, if a convolutional layer produces an 8-bit output, subsequent operations such as max-pooling and batch normalization are assigned the same precision, avoiding redundant higher-bit

computations. By systematically reducing bit-widths without compromising accuracy, this approach significantly decreases resource usage while maintaining high inference performance.

4.6 Design Space Exploration (DSE)

Within our approach, Design Space Exploration (DSE) algorithms are critical for automatically adjusting the optimization parameters in subsequent iterations to improve design performance, while maintaining high model accuracy and meeting the constraints of the device. We use Bayesian optimization to select high-level configuration parameters θ for the cross-stage flow, e.g., global tolerances $\tau = (\alpha_p, \alpha_s, \alpha_q)$. Layer-wise details are decided inside each O -task's inner search (Sections 4.2–4.4), while BO governs the outer loop (Algorithm 1).

4.6.1 Surrogate model. We model the scalarized objective $f(\theta)$ with a zero-mean Gaussian Process (GP) using a Matérn covariance function and homoscedastic Gaussian noise. Inputs are min-max normalized to $[0, 1]$. GP hyperparameters are re-estimated by maximizing the marginal likelihood at each iteration using the current design set.

4.6.2 Objective scalarization and normalization. Let results be Acc, DSP, LUT, Lat. We normalize each metric to $[0, 1]$ using ranges observed so far, then form the minimization objective

$$f(\theta) = -(w_1 \tilde{\text{Acc}} + w_2 \tilde{\text{DSP}} + w_3 \tilde{\text{LUT}} + w_4 \tilde{\text{Lat}})$$

subject to feasibility. Unless otherwise stated we use equal weights. To target particular frontiers we choose weights accordingly; e.g., Accuracy-DSP uses $(w_1, w_2, w_3, w_4) = (0.5, 0.5, 0, 0)$, Accuracy-LUT uses $(0.5, 0, 0.5, 0)$, and the balanced Acc-DSP-LUT-Lat setting uses $(0.25, 0.25, 0.25, 0.25)$.

4.6.3 Constraints. Feasibility requires: (i) device utilization within limits (DSP/LUT), (ii) latency $\leq t_{\max}$, and (iii) model accuracy loss $\leq \text{Acc_loss}_{\max}$. We treat constraints as *hard*: infeasible candidates are rejected and scored as $-M$ in Algorithm 1's Score function.

4.6.4 Acquisition. The Bayesian optimization algorithm finds next candidate in the search space by maximizing the acquisition function's value, where the acquisition function is Expected Improvement (EI).

4.6.5 Convergence. The outer BO loop stops when any of the following holds: (i) budget $T = 22$ iterations per ordering π , or (ii) the incumbent design remains unchanged for 5 iterations. Inner loops terminate as defined earlier: pruning uses a binary search until $|\Delta \text{rate}| < \beta_p$ (Section 4.2), scaling stops when loss exceeds α_s (Section 4.3), and QHS halts when no further reducible precisions remain under α_q (Section 4.4).

4.6.6 Global tolerance variables. We treat the vector of global tolerances $\tau = (\alpha_p, \alpha_s, \alpha_q)$ as part of the BO decision variable θ . The user supplies only an initial τ_0 and bounds $[\tau_{\min}, \tau_{\max}]$. At each BO iteration, the acquisition proposes a new θ_i that includes τ_i ; no per-benchmark manual tuning of α_x is required. Inner searches remain local to each O -task, while BO governs the outer loop over τ and flow-level choices (Algorithm 1).

4.7 DSE-Generated Hardware Architecture

We adopt a dataflow-style, layer-pipelined streaming hardware architecture, which is well-suited for implementing DNN designs. All candidates explored by MetaML-Pro are instantiated from the hls4ml hardware library as per-layer compute kernels (e.g., `nnet::conv_2d_c1`, and `nnet::dense`) connected by streaming channels. Concurrency (e.g., unrolling/reuse, initiation interval) and buffering (e.g., FIFO depths) are driven by HLS pragmas

Algorithm 1: MetaML-Pro: Cross-Stage Constrained BO with Bottom-Up Feedback and Order Search

```

1 Input: Optimization tasks  $O = \{\text{SCALING (S), PRUNING (P), QUANTIZATION (Q)}\}$ ;
   Orderings  $\Pi \subseteq \text{Perm}(O)$  (e.g.,  $\{S \rightarrow P \rightarrow Q, P \rightarrow S \rightarrow Q, \dots\}$ );
   Initial parameters  $\theta_1 = \tau_1 = (\alpha_p, \alpha_s, \alpha_q)$ ;
   Resource/latency/accuracy limits  $u_{\max}, t_{\max}, \text{acc\_loss}_{\max}$ ;
   BO budget  $T$ ; initial design set  $\mathcal{D}_0$ ; vendor set  $\mathcal{V} = \{\text{AMD, Intel}\}$ ;
   Meta-model  $MM$  with sections CFG/LOG/MODEL-SPACE.
Output: Best design(s)  $\mathcal{D}^*$  and Pareto set  $\mathcal{P}$ .

2 Initialize: Fit surrogate  $\mathcal{S}$  on  $\mathcal{D}_0$ ; Set BO results  $\mathcal{E}_\pi \leftarrow \emptyset$  for all  $\pi \in \Pi$ .
3 for  $\pi \in \Pi$  in parallel do // FORK over orderings
4   for  $t \leftarrow 1$  to  $T$  do
5      $m_t \leftarrow \text{Evaluate}(\pi, \theta_t, MM)$ 
6     if  $\text{Feasible}(m_t, \theta_t, u_{\max}, t_{\max}, \text{acc\_loss}_{\max})$  then
7        $s_t \leftarrow \text{Score}(\text{Normalize}(m_t))$ 
8     else
9        $s_t \leftarrow -M$  // constraint-violated candidate receives large negative score
10     $\mathcal{S} \leftarrow \text{UpdateSurrogate}(\mathcal{S}, \theta_t, s_t)$  // Fit surrogate
11     $\mathcal{E}_\pi \leftarrow \mathcal{E}_\pi \cup \{(\theta_t, m_t)\}$ 
12     $\theta_{t+1} \leftarrow \text{ProposeCandidate}(\mathcal{S}, \text{CFG}(MM), \pi)$  // acquisition-guided
13   $\mathcal{P} \leftarrow \text{ParetoReduce}(\bigcup_{\pi \in \Pi} \mathcal{E}_\pi)$  // REDUCE across paths
14   $\mathcal{D}^* \leftarrow \text{SelectBest}(\mathcal{P}, \text{policy})$ 
15 return  $(\mathcal{D}^*, \mathcal{P})$ 

16 Function  $\text{Evaluate}(\pi, \theta, MM)$ 
   // Top-down pass with inner DSE inside each O-task
17    $\text{Apply}(MM.\text{CFG}, \theta)$ 
18    $\text{Run}(\text{KERAS-MODEL-GEN})$ ;
19   foreach  $O \in \pi$  do
20      $\text{Run}(O)$  // SCALING/PRUNING/QUANT.: each has its own inner DSE
21    $\text{Run}(\text{HLS4ML})$ 
   // Vendor-aware BRANCH
22   if  $\text{VendorSelect}(MM) = \text{AMD}$  then
23      $\text{Run}(\text{VIVADO-HLS})$ 
24   else
25      $\text{Run}(\text{INTEL-oneAPI})$ 
26    $m \leftarrow \text{CollectMetrics}(\text{accuracy, latency, DSP, LUT, FF, BRAM})$ 
27   return  $m$ 

28 Function  $\text{Feasible}(m, \theta, u_{\max}, t_{\max}, \text{acc\_loss}_{\max})$ 
29   return  $(m.\text{util} \leq u_{\max})$  and  $(m.\text{lal} \leq t_{\max})$  and  $(m.\text{acc\_loss} \leq \text{acc\_loss}_{\max})$ 

30 Function  $\text{Score}(\tilde{m})$ 
31   return  $-(w_1 \tilde{m}.\text{Acc} + w_2 \tilde{m}.\text{DSP} + w_3 \tilde{m}.\text{LUT} + w_4 \tilde{m}.\text{lat})$ 

```

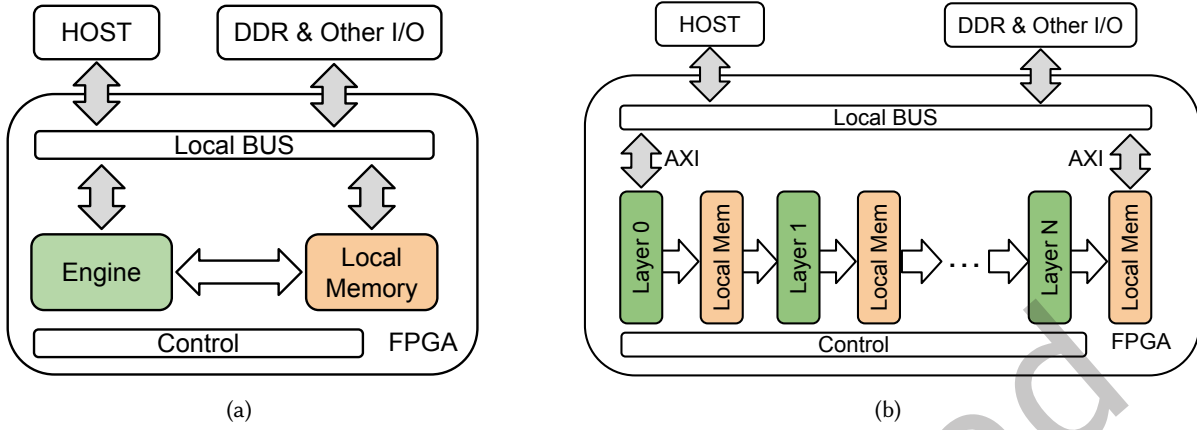


Fig. 8. FPGA-based DNN accelerator architectures: (a) A single-engine hardware architecture with a shared computation engine that is time-multiplexed across all layers; (b) A stream-based layer-wise pipeline hardware architecture with several custom engines, each processing a sub-layer or a whole layer or even multiple layers in a DNN model.

and layer configuration parameters. The library is initially configured via `IOType`, `default_precision`, `device`, and `clock` (Table 1). In addition, the generated HLS can be refined by metaprogramming that inserts/tunes pragmas and C++ codes through AST manipulation (Section 4.5).

This architecture adopts a layer-wise pipeline [28, 35, 37, 52], as shown in Fig. 8(b). Each major computational block of DNN models is mapped onto a tailored hardware module, with local memory connected via on-chip streams. The modules are chained into a deep pipeline so that different layers process different inputs concurrently, enabling continuous dataflow. In contrast to a conventional single-engine design [11, 41, 42] that time-multiplexes a common compute core across all layers (Fig. 8(a)), this architecture allows each block in DNN to be independently optimized for resource utilization, numerical precision, and micro-architecture. Intermediate data remain on-chip between layers to avoid repeated off-chip memory accesses and reduce inference latency, while exploiting the full potential of FPGA customizability. Please refer to [32] for a more detailed discussion of these two architectures.

In regards to the DSE process, our proposed Bayesian-optimization (BO) loop treats the global tolerance vector $\tau = (\alpha_p, \alpha_s, \alpha_q)$ as part of the decision variable and *automatically* proposes new τ_i each iteration. For each BO proposal, the O -tasks run inner searches that directly modify the hardware: (i) Pruning deletes weights/filters, shrinking convolutions and dense layers; (ii) Scaling reduces layer sizes; (iii) QUANTIZATION (QHS) assigns mixed-precision formats per virtual layer at the HLS C++ level. The outcome is an HLS-based C++ design with a specific level of parallelism, buffering, and custom numeric formats, ready for synthesis and evaluation.

4.8 Collaborative Optimization Mechanics

MetaML-Pro realizes collaboration across *four interacting layers* spanning software and HLS domains. First, it employs a data/state-level collaboration via the meta-model. All tasks (software and HLS) read/write a shared meta-model (CFG/LOG/MODEL-SPACE), which carries models, tool reports, and computed metrics across stages. This allows later-stage hardware outcomes (latency/utilization) to inform earlier software optimizations programmatically. Second, it employs control-level collaboration via K -tasks. BRANCH triggers bottom-up adaptation when constraints are violated; FORK/REDUCE explore alternative orderings and merge by Pareto dominance; JOIN composes multi-stage flows. This moves beyond static pipelines to *runtime* flow reconfiguration. Third, it has parameter-level collaboration via Bayesian Optimization based outer loop and inner local searches. The outer constrained BO jointly chooses the global tolerance vector $\tau = (\alpha_p, \alpha_s, \alpha_q)$ and searches across task orderings Π ; each O -task then runs its own inner DSE (binary-search pruning, progressive scaling, QHS quantization).

Table 2. Benchmarks used in this work

Model Name	Dataset	Application Domain
Jet-DNN [6, 9] / Jet-CNN	Jet-HLF [9]	Jet Identification (Particle Physics)
VGG-7 [51, 56]	MNIST [24]	Image Classification
ResNet-9 [13, 19]	SVHN [29]	Image Classification
LSTM [42]	MNIST	Sequence classification

This couples global, cross-stage choices with local decisions automatically. And finally, the HLS metaprogram edits C++ kernels (AST-driven insertion/tuning, data-type/reuse updates) and applies QHS with virtual layers, propagating precision constraints across cascaded operations.

5 Evaluation

In this section, we demonstrate how optimization strategies can be built by revising design-flow architectures, combining and reusing pipe tasks, and modifying their configuration (Sections 5.2 and 5.3). We explain flow control in Sections 5.6 and 5.5. We discuss optimisation search strategies and compare evaluation results with other approaches in Sections 5.7 and Section 5.10, respectively.

5.1 Experimental Setup

Experiments were conducted in Python 3.9.15 with benchmark workloads from typical DNN applications, as presented in Table 2), including jet identification [9, 27], image classification using VGG[51] and ResNet [19] networks. The jet identification (Jet-DNN) task targeted FPGA-based CERN Large Hadron Collider (LHC) triggers with a 40 MHz input rate and a response latency of less than 1 microsecond. Beyond Jet-DNN, we also evaluate VGG-7 (convolutional, MNIST), ResNet-9 (residual, SVHN), and LSTM (recurrent, sequential MNIST). These models are evaluated across single O-task strategies (pruning, quantization) and combined cross-stage strategies (e.g., $S \rightarrow P \rightarrow Q$), and we demonstrate portability across vendor toolchains (AMD/Xilinx and Intel). This breadth evidences that MetaML-Pro’s O-task library and K-task control (FORK/BANCH/REDUCE) generalize across CNN, residual, and RNN topologies and across vendor flows. Default frequencies were 100MHz for Zynq 7020 and 200MHz for Alveo U250 and VU9P, and the HLS4ML task used 18-bit fixed-point precision with 8 integer bits.

Although our quantitative evaluation uses Jet-DNN, VGG-7, ResNet-9, and LSTM, MetaML-Pro’s compute is governed by its controller and task structure rather than these particular networks. Concretely, the outer Bayesian Optimization (BO) loop runs with a fixed iteration budget (the number of iteration and orderings by default) and convergence criteria that are agnostic to network depth/width (Algorithm. 1), while each inner O-task scales gently with model size. Parallel FORK/REDUCE exploration and a thread-pool scheduler further amortize cost across orderings, and the meta-model caches intermediate artifacts across stages to avoid recomputation. These properties allow MetaML-Pro to scale to larger CNN and RNN models. We aim to extend our approach to emerging Transformer families in future work by plugging in the appropriate λ -tasks and kernels.

5.2 Single O-task Strategies

In this subsection, we focus on three strategies which are backed by a single O-task each, respectively.

5.2.1 Pruning strategy. The design flow for the pruning strategy is illustrated in Fig. 9(a) and the search steps and direction are shown in Fig. 10. Both the tolerance ($\leq \alpha_p$) and threshold (β_p) values are initially set to 2% in this strategy. The effectiveness of the auto-pruning algorithm is demonstrated in Fig. 11. Fig. 11 (a) and (c) depict the pruning rate and accuracy for Jet-DNN and ResNet9 in each step, while (b) and (d) show the resources

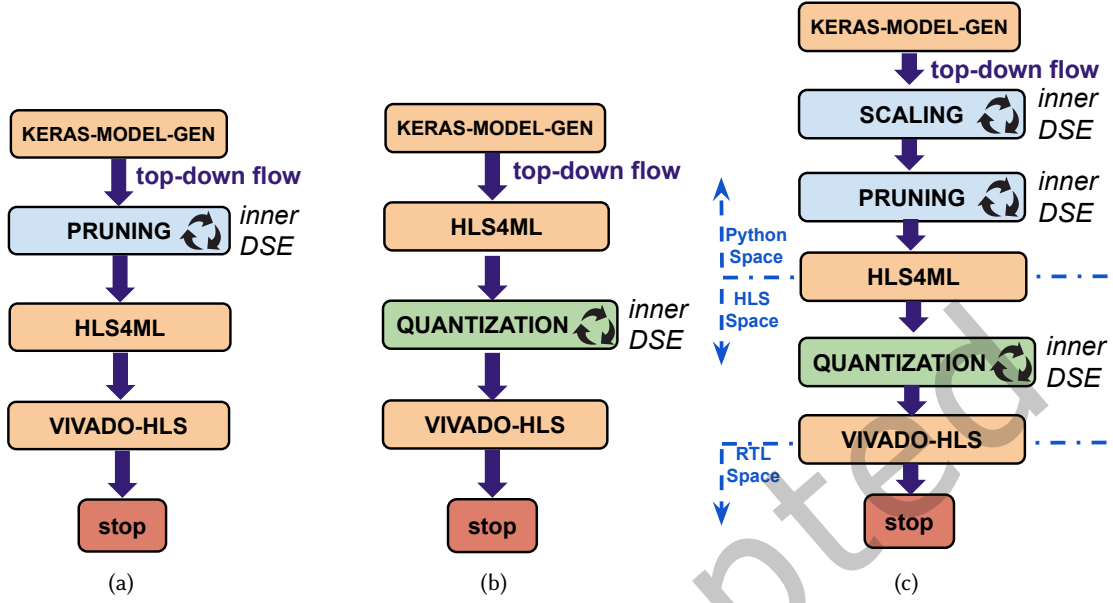


Fig. 9. (a) Pruning strategy. (b) Quantization strategy. (c) The combined strategy of scaling, pruning and quantization.

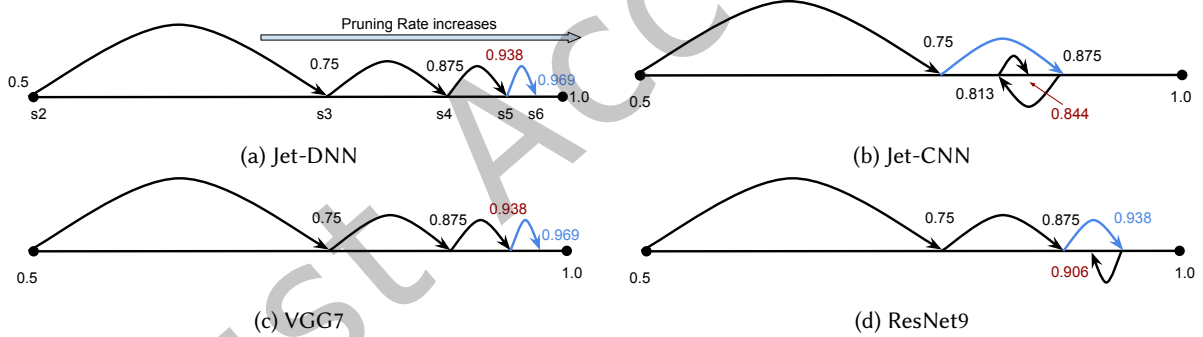


Fig. 10. The auto-pruning algorithm applied to models with binary search direction shown. Omitting step s1 for visibility. The blue arrow indicates an accuracy loss $>$ user threshold; red denotes the optimal pruning rate.

utilization on Zynq 7020 and U250. As the pruning rate increases, hardware resource requirements, particularly DSPs and LUTs, decrease, leading to improved FPGA performance. The design candidate with the highest pruning rate within the allowed tolerance is selected. In addition, Fig. 10(c)(d) shows that auto-pruning steadily reduces DSP/LUT utilization on U250 for ResNet-9 while maintaining accuracy within the configured α_p , mirroring the Jet-DNN trend and indicating robustness across CNN families.

5.2.2 Scaling strategy. To accommodate a large DNN design on an FPGA, we use the SCALING O -task that automatically reduces the layer size while tracking the accuracy loss α_s . The search stops either when the loss exceeds α_s . If necessary, α_s can be adjusted to achieve further size reduction with minimal impact on accuracy. This work sets α_s to 0.05%, which allows for model size reduction with negligible accuracy loss.

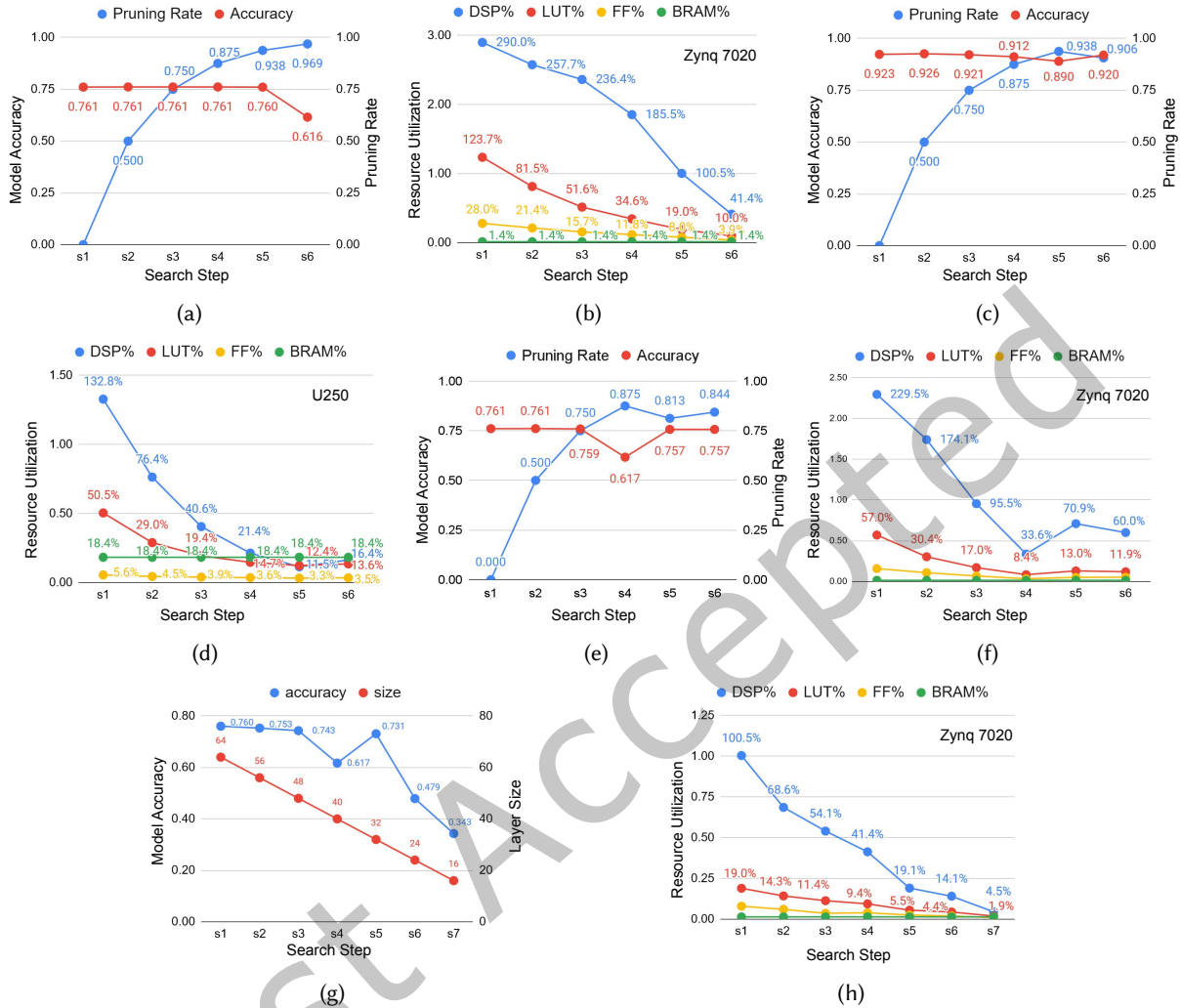


Fig. 11. (a) Pruning rates & accuracy of Jet-DNN. (b) Resource utilization of Jet-DNN design candidates after pruning. (c) Pruning rates & accuracy of ResNet9. (d) Resource utilization of ResNet9 design candidates after pruning. (e) Jet-DNN pruning rates & accuracy with scaling $\rightarrow$ pruning. (f) Resource utilization of Jet-DNN design candidates in (e). (g) Jet-DNN pruning rates & accuracy with pruning $\rightarrow$ scaling. (h) Resource utilization of Jet-DNN design candidates in (g).

5.2.3 Quantization strategy. This section showcases the evaluation results of the quantization (Q) optimization applied to multiple DNN models within the hardware (HLS) optimization space. The design flow for the quantization strategy is illustrated in Fig. 9(b).

Fig. 12 shows the precision of the weights, biases and output of each virtual layer of the VGG7 model after being tuned by the quantization strategy with α_q set to 1%. Table 3 shows how the quantization affected key evaluation metrics relating to the performance and resource usage of 2 DNN designs. With α_q set to 1%, the proposed QHS quantization algorithm reduces DSP usage by a factor of 4.9 for the VGG7 model, and 8.5 for the JetDNN model. When α_q is increased to 5%, the designs are further compacted, but with a larger real accuracy loss. This table highlights how varying α_q levels affect model accuracy, DSP, LUT, and FF usage, illustrating the

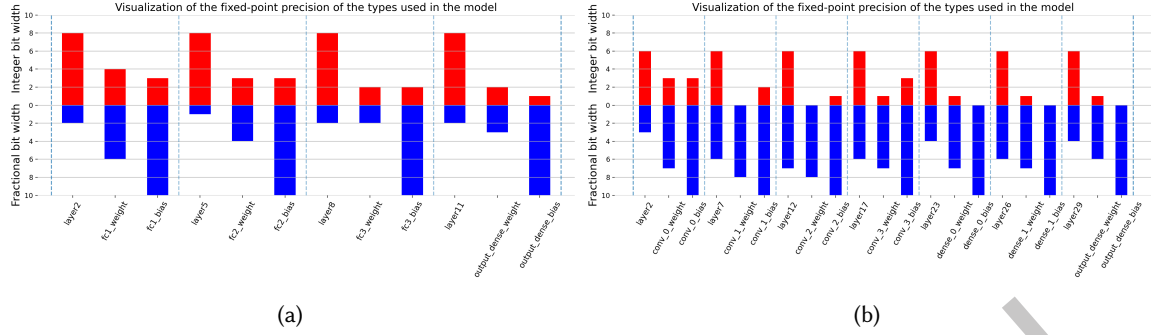


Fig. 12. (a) Quantized bitwidth of each layer in VGG7. (b) Quantized bitwidth of each layer in JetDNN.

Table 3. Results of designs with the quantization strategy using different α_q . VGG7 designs are using U250 while JetDNN designs are using Zynq 7020.

Model	α_q	DSP	LUT	FF	Acc
VGG7 Baseline	-	4568 (37.2%)	345k (20.0%)	65k (1.9%)	98.2%
VGG7 Quant.	0.01	934 (7.6%)	313k (18.1%)	51k (1.5%)	97.5%
VGG7 Quant.	0.05	505 (4.1%)	318k (18.4%)	66k (1.9%)	93.9%
JetDNN Baseline [36]	-	638 (290%)	66k (124%)	30k (28%)	76.1%
JetDNN Quant.	0.01	75 (34.1%)	57k (107%)	17k (16%)	75.5%
JetDNN Quant.	0.05	61 (27.7%)	76k (144%)	7.2k (6.8%)	71.6%

trade-offs between resource savings and accuracy as well as the effectiveness of the proposed QHS quantization algorithm.

5.3 Multiple O -task Strategies

With our framework, new strategies can be derived by building and revising a design-flow architecture. For instance, by inserting a scaling O -task before the pruning O -task in Fig. 9(a), a custom combined strategy, which conducts scaling-then-pruning (scaling $\rightarrow$ pruning), can be created with results shown in Fig. 11(e) and (f). The new optimal pruning rate is 84.4%, lower than the previous 93.8% (Fig. 11(a)), due to reduced redundancy from the preceding scaling task. By switching the order of the O -tasks, a different optimization strategy performing pruning-then-scaling (pruning $\rightarrow$ scaling) is achieved, resulting in a 0.7% accuracy drop after one scaling step, as seen in Fig. 11(g). Moreover, the three optimization O -tasks, pruning, scaling, and quantization, can be integrated into a single automated cross-stage strategy to enhance both performance and hardware efficiency, as illustrated in Fig. 9(c). We discuss the effects of different design-flow architectures with various combinations and orders in Section 5.7, and various tolerable loss in Section 5.8.

5.4 Branching Flow

Fig. 13(a) illustrates a design flow that performs pruning for two alternate targets using the BRANCH K -task, for AMD/Xilinx FPGAs and Intel FPGAs. A user-defined selection function is supplied as a parameter to the BRANCH K -task which encodes a strategy determining the path forward for the design.

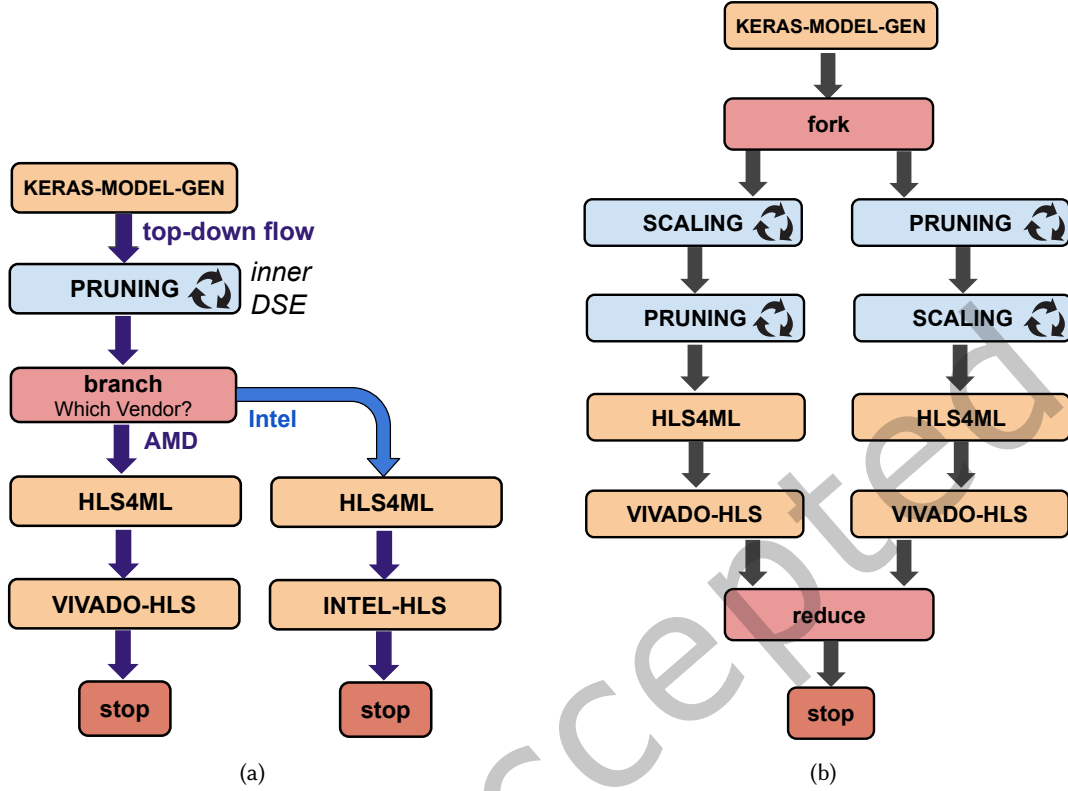


Fig. 13. (a) Pruning optimization targeting different vendors using the BRANCH K -task; (b) Combined strategy of scaling and pruning, exploring the order of O -tasks.

Fig. 14 presents the results of applying this design flow to an LSTM model on the MNIST dataset. Specifically, Fig. 14(a) illustrates the pruning rate and accuracy at each step using the PRUNING O -task. The tolerance is set to less than α_p (2%) in this design. Figs 14(b) and (c) show the resource utilization of the LSTM design after each pruning step respectively on an AMD KU115 FPGA and on an Intel A10 1150 FPGA. The DSP consumption is reduced from 6011 (108%) to 2101 (38.1%) on the AMD FPGA after the final pruning rate is optimized to be 71.9%. Compared to AMD's HLS compiler, which prefers DSP blocks, Intel's HLS compiler tends to favor the use of soft multipliers for implementation. As shown in Fig. 14(c), most of the computation kernels are implemented using logic resources rather than DSP blocks. It illustrates that the O -task ports across AMD/Intel with vendor-aware resource trade-offs.

While this design flow currently supports two types of FPGAs, it can be extended to include additional paths such as ASIC technologies. We leave the support for ASIC targets as our future work. Moreover, this evaluation underscores the flexibility of our approach in utilizing the same software optimization task, specifically PRUNING, across multiple hardware targets.

5.5 Parallel Flow

Our framework enables the execution of multiple optimization paths in parallel, allowing the selection of the best outcome among them. In Fig. 13(b), we show a design-flow with two parallel paths, where the execution order of two O -tasks is changed: scaling $\rightarrow$ pruning and pruning $\rightarrow$ scaling. To support parallel branches, we use the FORK task to connect multiple strategy paths. The results from each path are then evaluated based on predefined

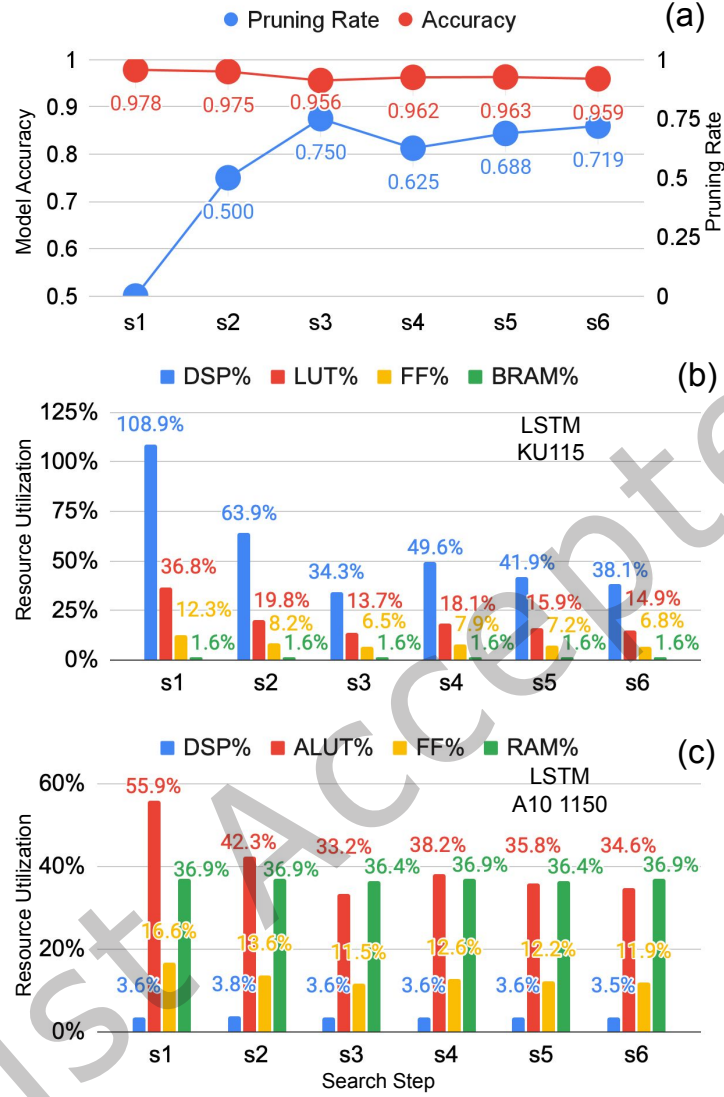


Fig. 14. LSTM model optimization on two FPGA platforms: (a) pruning rate and model accuracy using the PRUNING *O*-task; (b) resource utilization on an AMD KU115 FPGA; (c) resource utilization on an Intel A10 1150 FPGA.

criteria, such as accuracy or resource utilization, using the REDUCE task. For this strategy, a Pareto analysis was performed on the designs resulting from both paths, as shown in Fig. 15. By employing this design-flow, designers can explore various optimization combinations and sequences when the outcomes of these strategies are not clear.

5.6 Bottom-up Flow

Fig. 16 reveal two distinct flows in the optimization strategies: top-down, where information flows from the DNN to the hardware stage, and bottom-up, the reverse. We have automated both flows by customizing the *K*-task

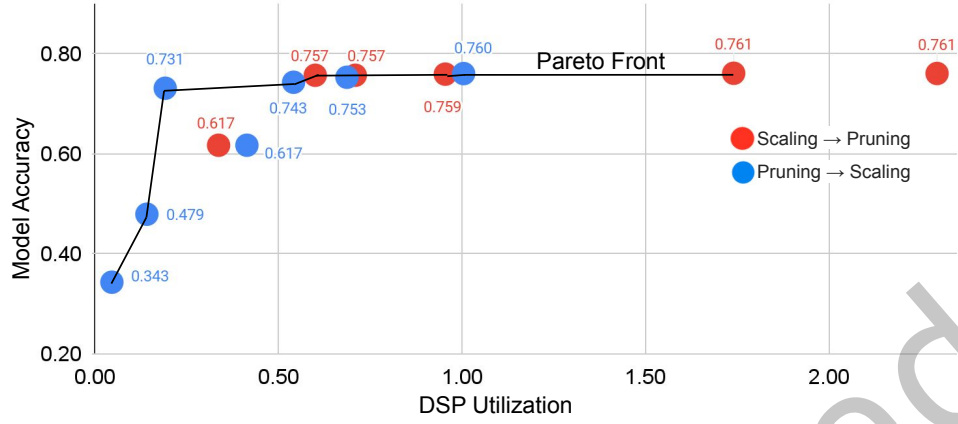
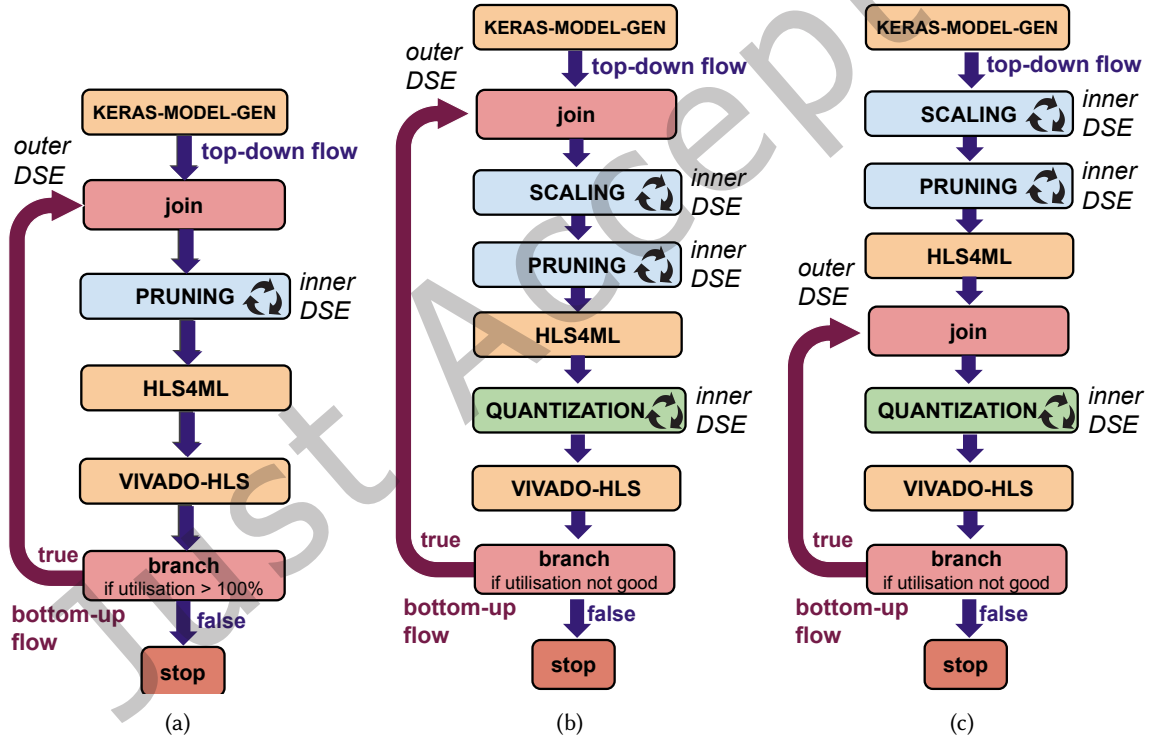
Fig. 15. Pareto Front meta-model designs post REDUCE K -task, color-coded paths.

Fig. 16. (a) Pruning strategy with Branch. (b) Combined strategy with Branch to python (SW) space. (c) The combined strategy with Branch to HLS space.

BRANCH with a user-defined predicate function to activate or stop the bottom-up flow if the resulting design overmaps. The BRANCH task also supports a user-supplied action function that is triggered when the predicate condition is true. For our current strategies, the action function changes the CFG section of the meta-model, increasing the accuracy tolerance parameters α_p , α_s , and α_q (if applicable) for the next iteration. Note that our

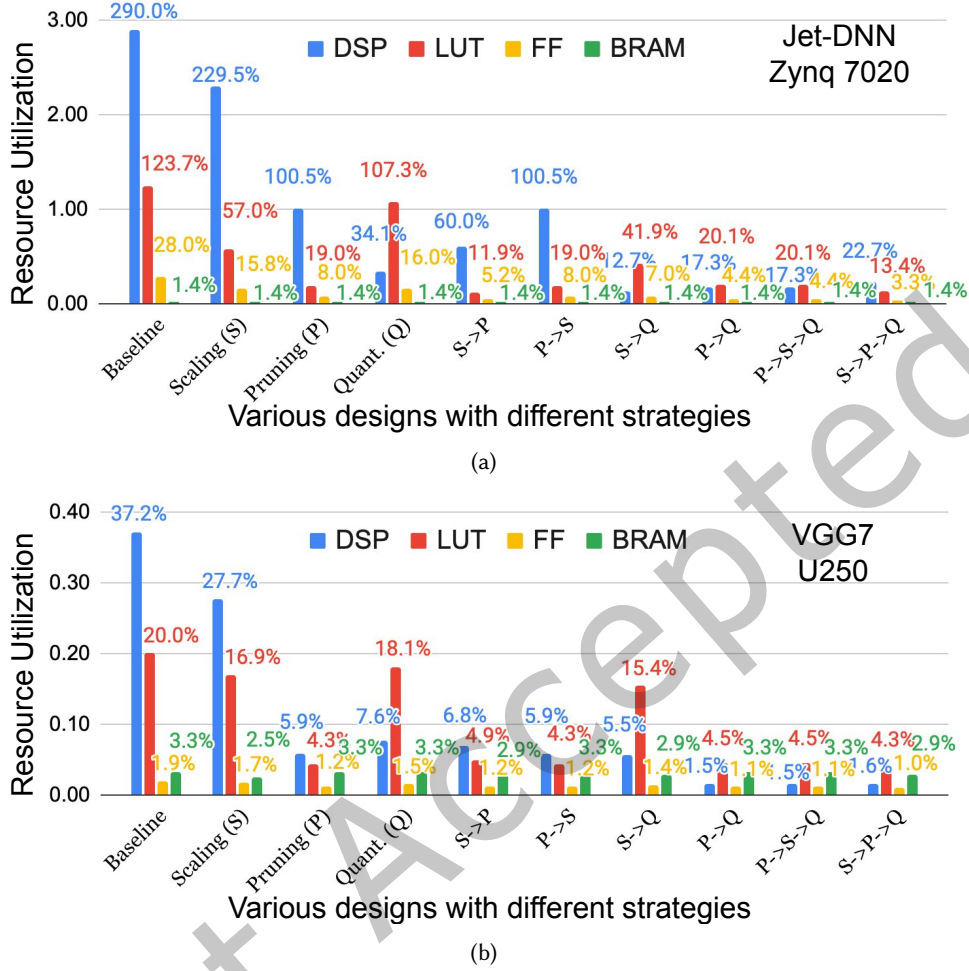


Fig. 17. (a) The hardware resources and latency of the Jet-DNN designs after various strategies. (b) The hardware resources and latency of the VGG7 designs after various strategies.

current strategies in Fig. 16 have two DSE loops: an inner-loop that is codified within each *O*-task and an outer-loop supported by the bottom-up flow. Users can develop more complex strategies by customizing the outer loop, for instance, by updating the bottom-up condition and parameter tuning.

5.7 Combination and Order

Combining optimization methods increases diversity and potential designs, improving optimal balance between accuracy and efficiency. We systematically evaluate all candidates combined optimization strategies based on our current set of 3 optimization tasks, Scaling (**S**), Pruning (**P**), and Quantization (**Q**), to identify the most effective combination and order. The α_p , α_s and α_q are set to 2%, 0.05% and 1%. In particular, Fig. 17(a) shows the hardware resource utilization results after each strategy and Fig 18 shows the corresponding latency, initiation interval and model accuracy. The final optimized model of Jet-DNN after scaling, pruning and quantization is depicted as “S→P→Q” design, resulting in a reduction of the DSP usage by around 92% and LUT usage by around 89% compared with the original design (baseline [9, 36]). In addition, the latency is reduced by 35% while the accuracy

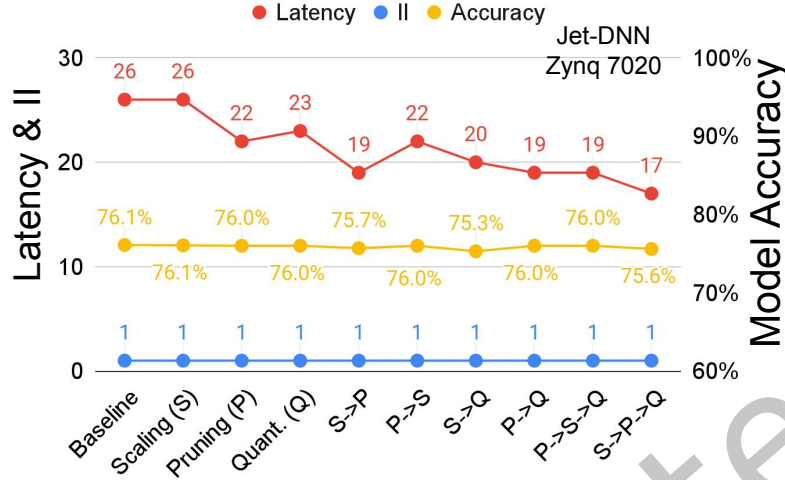


Fig. 18. Latency, Initiation Interval (II) and model accuracy of various designs with different strategies

loss is trivial, as shown in Fig. 18. The same search is performed on the VGG7 network with results shown in Fig. 17(b). The final design reduces DSP usage by a factor of 23 with the same latency and around 1.1% accuracy loss compared with the baseline design.

The search for optimal combinations and sequences with multiple O -tasks can be computationally demanding due to the exponential growth of the search space with the number of O -tasks. Advanced search methods can replace the current brute-force search. Recent studies [12, 22, 64] demonstrate surrogate models can expedite the search. Our framework facilitates the deployment of such techniques, enabling users to interface a search algorithm with the hardware design with minimal programming effort.

5.8 Tolerable Loss Variation

This subsection evaluates the performance and efficiency trade-offs when using different values for the maximum tolerable accuracy loss (α_p , α_s and α_q) for each optimization task to determine the effectiveness of their combination. We compare three strategies: Q , $S \rightarrow Q$, and $S \rightarrow P \rightarrow Q$, using Grid Search to create a Pareto frontier by assessing model accuracy against DSP and LUT utilization. As illustrated in Fig. 19, our findings reveal that the $S \rightarrow P \rightarrow Q$ strategy outperforms both the $S \rightarrow Q$ and Q strategies across multiple dimensions, including accuracy, DSP and LUT usage. However, it is important to note that the $S \rightarrow P \rightarrow Q$ strategy is less efficient in terms of time and design space requirements, demanding 220.5 times more time than the Q strategy. Practical applications may require a balance between performance, hardware resource utilization, and time investment, with the choice of strategy complexity and design space contingent on the specific use case. Our following experiments in this paper build on these findings, focusing on the $S \rightarrow P \rightarrow Q$ strategy.

5.9 Design-Space Exploration Strategies

Finding the optimal designs with grid search is time-consuming, as discussed in the previous section. This section investigates various DSE algorithms, including grid search, stochastic grid search (SGS), and Bayesian optimization. Fig. 20 presents the results. Each colored dot represents a design at a specific iteration, with different colors indicating different algorithms. The solid lines represent the Pareto Frontier of each algorithm over iterations, with each colored line corresponding to 22 iterations taking 40 hours. The uppermost grey line indicates a total of 343 iterations using extensive grid search over 624 hours, representing a baseline for

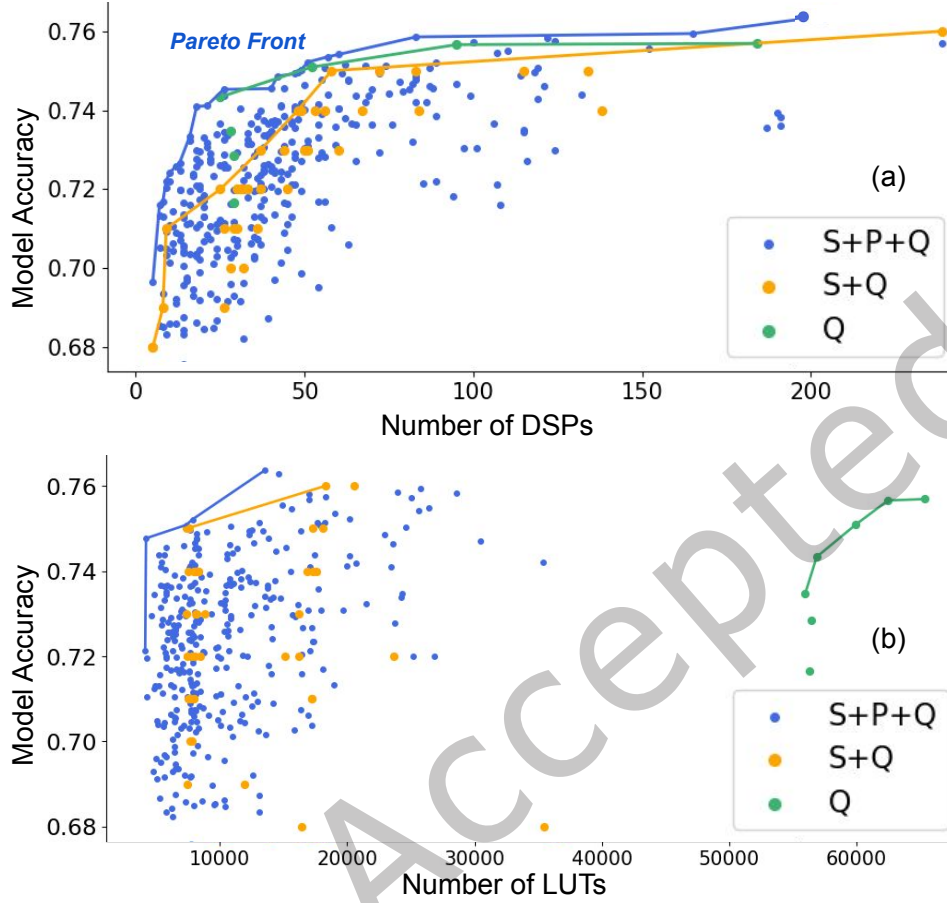


Fig. 19. Comparison of Pareto frontiers of **JetDNN** model accuracy and resource utilization using different optimization strategies.

comparison. The Bayesian optimization achieves similar results with just 22 iterations, significantly reducing processing time by a factor of 15.6 compared to Grid Search. Compared to SGS, Bayesian optimization's efficient parameter search yields multiple points near the Pareto frontier, indicating its effectiveness in finding optimal designs and approaching global optima. This effective parameter search by employing past results demonstrates the robustness of the Bayesian optimization approach, enabling users to optimize models while minimizing time and effort.

5.10 Discussion

5.10.1 Comparative Evaluation. Our evaluation results indicate that our combined *O*-task optimization strategy typically outperforms single *O*-task techniques. Furthermore, the order in which these optimization techniques are applied plays a crucial role, as different orders produce varying final results, as depicted in Fig. 17.

To highlight the advantages of our framework, we compare our results to those from other approaches targeting low-latency, low-resource, fully unfolded FPGA implementation of the JetDNN network, including LogicNets [58] JSC-M and JSC-L, QKeras-based Q6 [6], AutoQKeras-based QE and QB [6], and MetaML [36] in Fig. 21 and Table 4. All designs use the same architecture, except for JSC-L, which employs a larger architecture.

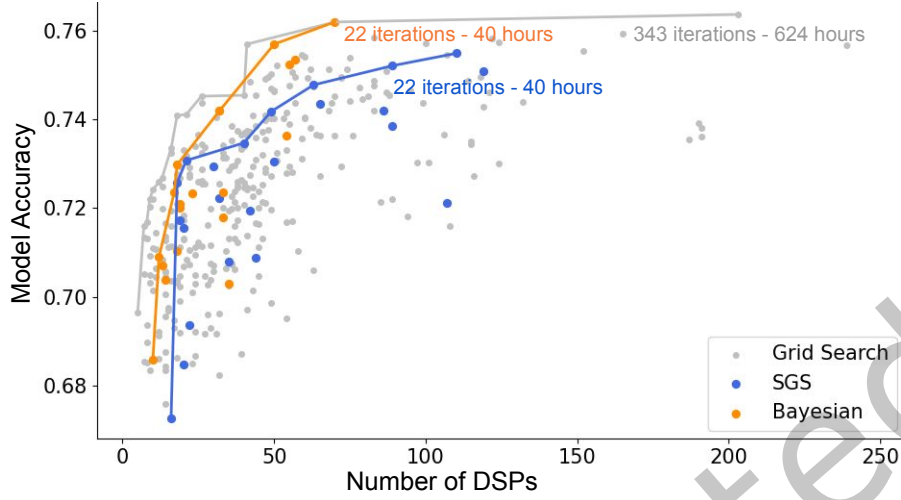


Fig. 20. DSP-Accuracy Pareto frontiers for each optimization using different DSE methods for JetDNN models.

Compared to original Jet-DNN [9], our design achieves higher accuracy (up to 76.2%) while using less hardware resources. When compared with LogicNets JSC-M and JSC-L [58], which achieve accuracies of 70.6% and 71.8% respectively, our design demonstrates up to 5.6% higher accuracy while also offering resource efficiency. Against the AutoQkeras Q6 and QE designs [6], which yield accuracies of 74.8% and 72.3% respectively, our framework attains higher accuracy and lower latency while providing more granular trade-offs between resource utilization and latency. Compared to MetaML [36] with manual optimization, our work achieves comparable or better accuracy across various configurations with automatic optimization. Overall, the versatility of our design, which balances accuracy, latency, and resource utilization across different objectives, highlights its superiority and adaptability for diverse FPGA-based DNN applications.

This effective parameter search demonstrates the robustness of the Bayesian Optimization approach, enabling users to optimize models while minimizing development time. Besides, this work demonstrates flexibility and effectiveness by achieving competitive performance across multiple configurations, highlighting its adaptability to different optimization priorities such as Accuracy, DSP and LUT. It is worth noting that our results are identified automatically while the other approaches involve manual optimization by DNN and hardware experts.

5.10.2 GNN: Scaling to graph-based models. Table 5 shows a case study on the GNN-based jet classifier of Odagiu et al. [31] with 8, 16, and 32 constituents on a Xilinx VU13P, where we apply the same PRUNING O -task with $\alpha_p = \beta_p = 2\%$. MetaML-Pro automatically finds configurations that prune 53.1–65.7% of the parameters, reducing DSP usage by up to 2.5 times and LUT usage by up to about 2 times, while improving latency from 160–205 ns to 145 ns and the initiation interval from 15 to 10 ns, with accuracy degradations within roughly 1–2 percentage points relative to the baselines. These results extend our evaluation to a larger, irregular graph-based model family.

5.10.3 Additional Exploration Methods. Beyond grid and Bayesian search, recent works explore learning-based and reinforcement learning (RL) strategies for accelerator DSE, as well as surrogate/ predictor-driven exploration. Examples include GNN- or RL-assisted HLS DSE and holistic AutoML-style co-design frameworks that learn fast proxies for hardware metrics to guide the search [22, 26, 48, 64, 69]. In our setting, these methods target the same goal as BO—efficiently proposing promising design points under expensive evaluations—but differ in how they trade off sample efficiency, parallelism, and hyperparameter overhead. Our framework is compatible

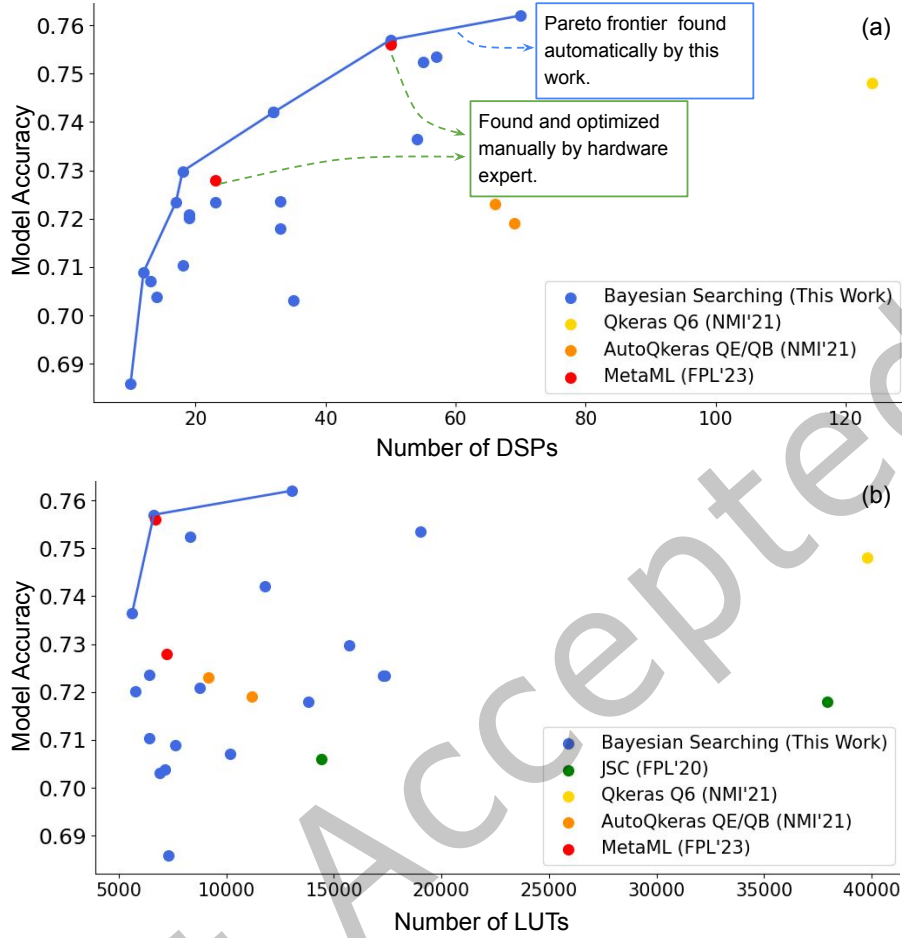


Fig. 21. Comparison of resource utilization of the FPGA-based **JetDNN** networks using our approach and others (LogicNets JSC [58], Qkeras Q6 [6], AutoQkeras QE/QB [6] and MetaML [36]) on an AMD/Xilinx VU9P FPGA. The Pareto frontier is highlighted.

with these alternatives: the outer DSE loop interfaces our cross-stage O-tasks and K-task controls through the meta-model (CFG/LOG/MODEL-SPACE), so an evolutionary or RL agent can be substituted for BO without changing inner optimizations or the flow architecture.

5.10.4 Extensibility of the Framework. While this paper instantiates three SW-level O-tasks (SCALING, PRUNING) and an HLS-level mixed-precision QUANTIZATION (QHS), the pipe-task API and cross-stage controller are agnostic to the specific algorithm. The other optimization, such as hardware-aware NAS, knowledge distillation, structured sparsity and RTL level optimizations can be added as drop-in O-tasks/K-tasks that participate in the same FORK/BRANCH/REDUCE and BO-driven outer loop without changing the framework. In addition to new O/K-tasks, the outer DSE module is swappable: evolutionary or RL-based search can be plugged in as the controller that proposes (τ, Π) , orchestrating the same FORK/BRANCH/REDUCE flow and inner searches without changing the pipeline code.

Table 4. Performance comparison with the FPGA designs of Jet-DNN network using other approaches on Xilinx FPGAs with a clock frequency of 200MHz. Please note that this table only reports Jet-DNN to enable an apples-to-apples comparison with prior Jet-DNN FPGA designs.

Model	$\alpha_s, \alpha_p, \alpha_q$ (%)	FPGA	Acc.(%)	Lat. (ns)	DSP (%)	LUT (%)	Pow. (W) ^e
HLS4ML Jet-DNN [9]	-	KU115	75	75	954 (17.3)	-	-
FPL'20 LogicNets JSC-M [58]	-	VU9P	70.6	NA	0 (0)	14,428 (1.2)	-
FPL'20 LogicNets JSC-L [58]	-	VU9P	71.8	13 ^a	0 (0)	37,931 (3.2)	-
NMI'21 [6] Qkeras Q6	-	VU9P	74.8	55	124 (1.8)	39,782 (3.4)	-
NMI'21 [6] AutoQkeras QE	-	VU9P	72.3	55	66 (1.0)	9,149 (0.8)	-
NMI'21 [6] AutoQkeras QB	-	VU9P	71.9	70	69 (1.0)	11,193 (0.9)	-
FPL'23 [36] MetaML	-	VU9P	76.1	70	638 (9.3)	69,751 (5.9)	2.506
FPL'23 [36] MetaML	{0.05, 2, 1}	VU9P	75.6	45	50 (0.7)	6,698 (0.6)	0.199
FPL'23 [36] MetaML	{0.05, 2, 4}	VU9P	72.8	40	23 (0.2)	7,224 (0.6)	0.169
This work (Best Acc.)	{0.5, 0.1, 0.1}	VU9P	76.2	55	272 (4.0)	14,580 (1.2)	1.096
This work (Best DSP)	{0.5, 3, 4}	VU9P	69.7	40	5 (0.1)	9,026 (0.8)	0.118
This work (Best LUT)	{2, 5, 1}	VU9P	72.1	45	47 (0.7)	4,410 (0.4)	0.204
This work ^b (Acc.-DSP-LUT)	{0.5, 2, 0.5}	VU9P	76.1	50	70 (1.0)	13,042 (1.1)	0.386
This work ^b (Acc.-DSP-LUT)	{4, 0.5, 0.5}	VU9P	75.7	45	50 (0.7)	6,634 (0.6)	0.247
This work ^c (Acc.-LUT)	{4, 4, 0.5}	VU9P	73.7	45	54 (0.8)	5,624 (0.5)	0.246
This work ^d (Acc.-DSP)	{2, 3, 4}	VU9P	70.9	40	12 (0.2)	7,637 (0.8)	0.130

^a A clock frequency of 384 MHz is used and the final softmax layer is removed.

^b On both Accuracy-DSP and Accuracy-LUT Pareto lines as shown in Fig. 21.

^c Only on Accuracy-LUT Pareto line. ^d Only on Accuracy-DSP Pareto line.

^e Dynamic power consumption reported by Vivado out-of-context (OOC) design flow. The static power is about 2.5W for all these designs.

Table 5. Performance comparison with FPGA designs of GNN networks with 8/16/32 constituents on Xilinx VU13P FPGAs with a clock frequency of 200MHz. The pruning strategy is applied with both α_p and β_p set to 2%.

Model	FPGA	Pru. (%)	Acc.(%)	Lat. (ns)	II (ns)	DSP (%)	LUT (%)	Pow. (W) ^a
GNN-8C [31]	VU13P	0	64.9	160	15	2,191 (17.8)	472k (27.3)	-
GNN-16C [31]	VU13P	0	70.8	180	15	5,362 (43.6)	1,387k (80.3)	-
GNN-32C [31]	VU13P	50	75.8	205	15	2,120 (17.3)	1,162k (67.3)	-
This work, GNN-8C	VU13P	65.7	63.2	145	10	1,005 (8.2)	240k (13.9)	5.123
This work, GNN-16C	VU13P	53.1	68.9	145	10	5,013 (40.8)	896k (51.9)	10.076
This work, GNN-32C	VU13P	62.5	74.6	145	10	845 (6.9)	1,017k (58.8)	9.177

^a On-chip power consumption estimated using Xilinx Vivado 2020.1 `report_power` in out-of-context (OOC) mode at 200 MHz, core logic only.

6 Conclusion

This paper presents a novel co-optimization framework for FPGA-based DNN accelerators, which comprises building blocks that facilitate rapid development of customized cross-stage design flows, automating the entire design iteration process. The results demonstrate that our approach significantly reduces DSP resource usage by up to 92% and LUT usage by up to 89%, while maintaining accuracy and without requiring human effort or domain

expertise. In addition, our results reveal that Bayesian optimization in DSE significantly streamlines the process, achieving results comparable to grid search but with a 15.6-fold reduction in processing time. Future work will extend this approach to include RTL space and ASIC platforms, support a broader range of DNN architectures, including Variational Autoencoder (VAE) [45] and large language models (LLMs) [7], and incorporate more optimization strategies like balancing initiation interval [44, 49] and distributed arithmetic (DA) [54] for further improvements in hardware efficiency and model performance.

Acknowledgement: The support of the United Kingdom EPSRC (grant number EP/V028251/1, EP/N031768/1, EP/S030069/1, and EP/X036006/1), UKRI256, KIAT, Intel, and AMD is gratefully acknowledged. We thank Markus Rognlien, Shuo Liu and Anyan Zhao for their help.

References

- [1] 2023. OpenVINO toolkit. <https://github.com/openvinotoolkit/openvino>.
- [2] Mohamed S Abdelfattah, Lukasz Dudziak, Thomas Chau, Royson Lee, Hyeji Kim, and Nicholas D Lane. 2020. Best of both worlds: Automl codesign of a cnn and its hardware accelerator. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [3] Nicolas Bohm Agostini, Serena Curzel, Vinay Amatya, Cheng Tan, Marco Minutoli, Vito Giovanni Castellana, Joseph Manzano, David Kaeli, and Antonino Tumeo. 2022. An MLIR-based Compiler Flow for System-Level Design and Hardware Acceleration. In *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*. 1–9.
- [4] Norman Carver and Victor Lesser. 1994. Evolution of blackboard control architectures. *Expert systems with applications* 7, 1 (1994), 1–30.
- [5] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*. 578–594.
- [6] Claudionor N Coelho, Aki Kuusela, Shan Li, Hao Zhuang, Jennifer Ngadiuba, Thea Klæboe Aarrestad, Vladimir Loncar, Maurizio Pierini, Adrian Alan Pol, and Sioni Summers. 2021. Automatic heterogeneous quantization of deep neural networks for low-latency inference on the edge for particle detectors. *Nature Machine Intelligence* 3, 8 (2021), 675–686.
- [7] Jorge R De Freitas, Jose GF Coutinho, Ce Guo, Suleyman Demirsoy, Wayne Luk, and Zhiqiang Que. 2025. Optimizing LLM inference for FPGAs. In *IEEE 16th International Conference on ASIC (ASICON)* (Accepted).
- [8] Zhen Dong, Yizhao Gao, Qijing Huang, John Wawrzynek, Hayden KH So, and Kurt Keutzer. 2021. Hao: Hardware-aware neural architecture optimization for efficient inference. In *2021 IEEE 29th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 50–59.
- [9] Javier Duarte, Song Han, Philip Harris, Sergio Jindariani, Edward Kreinar, Benjamin Kreis, Jennifer Ngadiuba, Maurizio Pierini, Ryan Rivera, Nhan Tran, et al. 2018. Fast inference of deep neural networks in FPGAs for particle physics. *Journal of Instrumentation* 13, 07 (2018), P07027.
- [10] Hongxiang Fan, Martin Ferianc, Zhiqiang Que, He Li, Shuanglong Liu, Xinyu Niu, and Wayne Luk. 2022. Algorithm and hardware co-design for reconfigurable cnn accelerator. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 250–255.
- [11] Hongxiang Fan, Shuanglong Liu, Zhiqiang Que, Xinyu Niu, and Wayne Luk. 2021. High-performance acceleration of 2-D and 3-D CNNs on FPGAs using static block floating point. *IEEE Transactions on Neural Networks and Learning Systems* 34, 8 (2021), 4473–4487.
- [12] Lorenzo Ferretti, Andrea Cini, Georgios Zacharopoulos, Cesare Alippi, and Laura Pozzi. 2022. Graph Neural Networks for High-Level Synthesis Design Space Exploration. *ACM Transactions on Design Automation of Electronic Systems* 28, 2 (2022), 1–20.
- [13] Fumio Hamanaka, Takashi Odan, Kenji Kise, and Thiem Van Chu. 2023. An Exploration of State-of-the-art Automation Frameworks for FPGA-based DNN Acceleration. *IEEE Access* (2023).
- [14] Cong Hao and Deming Chen. 2018. Deep neural network model and FPGA accelerator co-design: Opportunities and challenges. In *2018 14th IEEE International Conference on Solid-State and Integrated Circuit Technology (ICSICT)*. IEEE, 1–4.
- [15] Cong Hao, Yao Chen, Xinheng Liu, Atif Sarwari, Daryl Sew, Ashutosh Dhar, Bryan Wu, Dongdong Fu, Jinjun Xiong, Wen-mei Hwu, et al. 2019. NAIS: Neural architecture and implementation search and its applications in autonomous driving. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*. IEEE, 1–8.
- [16] Cong Hao, Yao Chen, Xiaofan Zhang, Yuhong Li, Jinjun Xiong, Wen-mei Hwu, and Deming Chen. 2020. Effective algorithm-accelerator co-design for ai solutions on edge devices. In *Proceedings of the 2020 on Great Lakes Symposium on VLSI*. 283–290.
- [17] Cong Hao, Jordan Dotzel, Jinjun Xiong, Luca Benini, Zhiru Zhang, and Deming Chen. 2021. Enabling design methodologies and future trends for edge AI: specialization and codesign. *IEEE Design & Test* 38, 4 (2021), 7–26.

- [18] Cong Hao, Xiaofan Zhang, Yuhong Li, Sitao Huang, Jinjun Xiong, Kyle Rupnow, Wen-mei Hwu, and Deming Chen. 2019. FPGA/DNN co-design: An efficient design methodology for IoT intelligence on the edge. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
- [19] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [20] Weiwen Jiang, Lei Yang, Edwin Hsing-Mean Sha, Qingfeng Zhuge, Shouzheng Gu, Sakyasingha Dasgupta, Yiyu Shi, and Jingtong Hu. 2020. Hardware/software co-exploration of neural architectures. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 4805–4815.
- [21] Vinod Kathail. 2020. Xilinx vitis unified software platform. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 173–174.
- [22] Maciej Kurek, Marc Peter Deisenroth, Wayne Luk, and Timothy Todman. 2016. Knowledge transfer in automatic optimisation of reconfigurable designs. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 84–87.
- [23] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. 2015. Deep learning. *nature* 521, 7553 (2015), 436–444.
- [24] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. 1998. Gradient-based learning applied to document recognition. *Proc. IEEE* 86, 11 (1998), 2278–2324.
- [25] Binglei Lou, Gautham Duddi Krishnaswaroop, Filip Wojcicki, Ruilin Wu, Richard Rademacher, Zhiqiang Que, Wayne Luk, and Philip HW Leong. 2025. Low-Latency FPGA Control System for Real-Time Neural Network Processing in CCD-Based Trapped-Ion Qubit Measurement. *arXiv preprint arXiv:2512.15838* (2025).
- [26] Atefeh Mehrabi, Aninda Manocha, Benjamin C Lee, and Daniel J Sorin. 2020. Bayesian optimization for efficient accelerator synthesis. *ACM Transactions on Architecture and Code Optimization (TACO)* 18, 1 (2020), 1–25.
- [27] Eric A Moreno, Olmo Cerri, Javier M Duarte, Harvey B Newman, Thong Q Nguyen, Avikar Periwal, Maurizio Pierini, Aidana Serikova, Maria Spiropulu, and Jean-Roch Vlimant. 2020. JEDI-net: a jet identification algorithm based on interaction networks. *The European Physical Journal C* 80, 1 (2020), 1–15.
- [28] Hiroki Nakahara, Zhiqiang Que, and Wayne Luk. 2020. High-throughput convolutional neural network on an FPGA by customized JPEG compression. In *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE Computer Society, 1–9.
- [29] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. 2011. Reading digits in natural images with unsupervised feature learning. (2011).
- [30] Jonas Ney, Dominik Lorocho, Vladimir Rybalkin, Nico Weber, Jens Krüger, and Norbert Wehn. 2021. HALF: Holistic auto machine learning for FPGAs. In *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 363–368.
- [31] Patrick Odagiu, Zhiqiang Que, Javier Duarte, Johannes Haller, Gregor Kasieczka, Artur Lobanov, Vladimir Loncar, Wayne Luk, Jennifer Ngadiuba, Maurizio Pierini, et al. 2024. Ultrafast jet classification at the HL-LHC. *Machine Learning: Science and Technology* 5, 3 (2024), 035017.
- [32] Zhiqiang Que. 2023. Reconfigurable acceleration of Recurrent Neural Networks. *PhD dissertation* (2023).
- [33] Zhiqiang Que, Jose GF Coutinho, and Wayne Luk. 2024. Deep Learning Design-Flow with Static and Dynamic Optimizations. In *2024 IEEE 17th International Conference on Solid-State & Integrated Circuit Technology (ICSICT)*. IEEE, 1–4.
- [34] Zhiqiang Que, Hongxiang Fan, Gabriel Figueiredo, Ce Guo, Wayne Luk, Ryota Yasudo, and Masato Motomura. 2025. Trustworthy Deep Learning Acceleration with Customizable Design Flow Automation. In *Proceedings of the 15th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies*. 1–13.
- [35] Zhiqiang Que, Hongxiang Fan, Marcus Loo, He Li, Michaela Blott, Maurizio Pierini, Alexander Tapper, and Wayne Luk. 2024. LL-GNN: Low latency graph neural networks on FPGAs for high energy physics. *ACM Transactions on Embedded Computing Systems* 23, 2 (2024), 1–28.
- [36] Zhiqiang Que, Shuo Liu, Markus Rognlien, Ce Guo, Jose GF Coutinho, and Wayne Luk. 2023. MetaML: Automating Customizable Cross-Stage Design-Flow for Deep Learning Acceleration. In *2023 30th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE.
- [37] Zhiqiang Que, Marcus Loo, Hongxiang Fan, Maurizio Pierini, Alexander Tapper, and Wayne Luk. 2022. Optimizing graph neural networks for jet tagging in particle physics on FPGAs. In *2022 32nd International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 327–333.
- [38] Zhiqiang Que, Marcus Loo, and Wayne Luk. 2022. Reconfigurable acceleration of graph neural networks for jet identification in particle physics. In *2022 IEEE 4th International Conference on Artificial Intelligence Circuits and Systems (AICAS)*. IEEE, 202–205.
- [39] Zhiqiang Que, Hiroki Nakahara, Hongxiang Fan, He Li, Jiuxi Meng, Kuen Hung Tsoi, Xinyu Niu, Eriko Nurvitadhi, and Wayne Luk. 2022. Remann: A reconfigurable multi-threaded multi-core accelerator for recurrent neural networks. *ACM Transactions on Reconfigurable Technology and Systems* 16, 1 (2022), 1–26.

- [40] Zhiqiang Que, Hiroki Nakahara, Hongxiang Fan, Jiuxi Meng, Kuen Hung Tsoi, Xinyu Niu, Eriko Nurvitadhi, and Wayne Luk. 2020. A reconfigurable multithreaded accelerator for recurrent neural networks. In *2020 International conference on field-programmable technology (ICFPT)*. IEEE, 20–28.
- [41] Zhiqiang Que, Hiroki Nakahara, Eriko Nurvitadhi, Andrew Boutros, Hongxiang Fan, Chenglong Zeng, Jiuxi Meng, Kuen Hung Tsoi, Xinyu Niu, and Wayne Luk. 2021. Recurrent neural networks with column-wise matrix–vector multiplication on FPGAs. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 30, 2 (2021), 227–237.
- [42] Zhiqiang Que, Hiroki Nakahara, Eriko Nurvitadhi, Hongxiang Fan, Chenglong Zeng, Jiuxi Meng, Xinyu Niu, and Wayne Luk. 2020. Optimizing Reconfigurable Recurrent Neural Networks. In *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 10–18.
- [43] Zhiqiang Que, Chang Sun, Sudarshan Paramesvaran, Emyr Clement, Katerina Karakoulaki, Christopher Brown, Lauri Laatu, Arianna Cox, Alexander Tapper, Wayne Luk, and Maria Spiropulu. 2025. JEDI-linear: Fast and Efficient Graph Neural Networks for Jet Tagging on FPGAs. In *2025 International Conference on Field Programmable Technology (FPT)*. IEEE.
- [44] Zhiqiang Que, Erwei Wang, Umar Marikar, Eric Moreno, Jennifer Ngadiuba, Hamza Javed, Bartłomiej Borzyszkowski, Thea Aarrestad, Vladimir Loncar, Sioni Summers, et al. 2021. Accelerating recurrent neural networks for gravitational wave experiments. In *2021 IEEE 32nd International Conference on Application-specific Systems, Architectures and Processors (ASAP)*. IEEE, 117–124.
- [45] Zhiqiang Que, Minghao Zhang, Hongxiang Fan, He Li, Ce Guo, and Wayne Luk. 2024. Low latency variational autoencoder on FPGAs. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 14, 2 (2024), 323–333.
- [46] Zhiqiang Que, Anyan Zhao, Jose GF Coutinho, Ce Guo, and Wayne Luk. 2024. Optimizing DNN Accelerator Compression Using Tolerable Accuracy Loss. In *International conference on field-programmable technology (FPT)*. IEEE.
- [47] Zhiqiang Que, Yongxin Zhu, Hongxiang Fan, Jiuxi Meng, Xinyu Niu, and Wayne Luk. 2020. Mapping large LSTMs to FPGAs with weight reuse. *Journal of Signal Processing Systems* 92, 9 (2020), 965–979.
- [48] Brandon Reagen, José Miguel Hernández-Lobato, Robert Adolf, Michael Gelbart, Paul Whatmough, Gu-Yeon Wei, and David Brooks. 2017. A case for efficient accelerator design space exploration via bayesian optimization. In *2017 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. IEEE, 1–6.
- [49] Markus Rognlien, Zhiqiang Que, Jose GF Coutinho, and Wayne Luk. 2022. Hardware-aware optimizations for deep learning inference on edge devices. In *International Symposium on Applied Reconfigurable Computing*. Springer, 118–133.
- [50] Jan-Frederik Schulte, Benjamin Ramhorst, Chang Sun, Jovan Mitrevski, Nicolò Ghielmetti, Enrico Lupi, Dimitrios Danopoulos, Vladimir Loncar, Javier Duarte, David Burnette, et al. 2025 (Accepted). hls4ml: A Flexible, Open-Source Platform for Deep Learning Acceleration on Reconfigurable Hardware. *ACM Transactions on Reconfigurable Technology and Systems* (2025 (Accepted)).
- [51] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
- [52] Chang Sun, Jennifer Ngadiuba, Maurizio Pierini, and Maria Spiropulu. 2025. Fast Jet Tagging with MLP-Mixers on FPGAs. *Machine Learning: Science and Technology* (2025).
- [53] Chang Sun, Zhiqiang Que, Thea K Aarrestad, Vladimir Loncar, Jennifer Ngadiuba, Wayne Luk, and Maria Spiropulu. 2026. HGQ: High Granularity Quantization for Real-time Neural Networks on FPGAs. (2026).
- [54] Chang Sun, Zhiqiang Que, Vladimir Loncar, Wayne Luk, and Maria Spiropulu. 2025. da4ml: Distributed arithmetic for real-time neural networks on fpgas. *ACM Transactions on Reconfigurable Technology and Systems* (2025).
- [55] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S Emer. 2017. Efficient processing of deep neural networks: A tutorial and survey. *Proc. IEEE* 105, 12 (2017), 2295–2329.
- [56] Stephen Tridgell, Martin Kumm, Martin Hardieck, David Boland, Duncan Moss, Peter Zipf, and Philip HW Leong. 2019. Unrolling Ternary Neural Networks. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 12, 4 (2019), 1–23.
- [57] Shikhar Tuli, Chia-Hao Li, Ritvik Sharma, and Niraj K Jha. 2023. CODEBench: A neural architecture and hardware accelerator co-design framework. *ACM Transactions on Embedded Computing Systems* 22, 3 (2023), 1–30.
- [58] Yaman Umuroglu, Yash Akhauri, Nicholas James Fraser, and Michaela Blott. 2020. LogicNets: co-designed neural networks and circuits for extreme-throughput applications. In *2020 30th International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 291–297.
- [59] Yaman Umuroglu, Nicholas J Fraser, Giulio Gambardella, Michaela Blott, Philip Leong, Magnus Jahre, and Kees Vissers. 2017. FINN: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA international symposium on field-programmable gate arrays*. 65–74.
- [60] Jessica Vandebon, Jose G. F. Coutinho, Wayne Luk, Eriko Nurvitadhi, and Tim Todman. 2020. Artisan: a Meta-Programming Approach For Codifying Optimisation Strategies. In *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*.
- [61] Stylianos I Venieris and Christos-Savvas Bouganis. 2016. fpgaConvNet: A framework for mapping convolutional neural networks on FPGAs. In *2016 IEEE 24th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 40–47.

- [62] Erwei Wang, James J Davis, Ruizhe Zhao, Ho-Cheung Ng, Xinyu Niu, Wayne Luk, Peter YK Cheung, and George A Constantinides. 2019. Deep neural network approximation for custom hardware: Where we’ve been, where we’re going. *ACM Computing Surveys (CSUR)* 52, 2 (2019), 1–39.
- [63] Filip Wojcicki, Zhiqiang Que, Alexander D Tapper, and Wayne Luk. 2022. Accelerating transformer neural networks on fpgas for high energy physics experiments. In *2022 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 1–8.
- [64] Nan Wu, Yuan Xie, and Cong Hao. 2021. Ironman: GNN-assisted design space exploration in high-level synthesis via reinforcement learning. In *Proceedings of the 2021 on Great Lakes Symposium on VLSI*. 39–44.
- [65] Pengfei Xu, Xiaofan Zhang, Cong Hao, Yang Zhao, Yongan Zhang, Yue Wang, Chaojian Li, Zetong Guan, Deming Chen, and Yingyan Lin. 2020. AutoDNNchip: An automated DNN chip predictor and builder for both FPGAs and ASICs. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 40–50.
- [66] Yifan Yang, Qijing Huang, Bichen Wu, Tianjun Zhang, Liang Ma, Giulio Gambardella, Michaela Blott, Luciano Lavagno, Kees Visser, John Wawrzyniec, et al. 2019. Synetgy: Algorithm-hardware co-design for convnet accelerators on embedded fpgas. In *Proceedings of the 2019 ACM/SIGDA international symposium on field-programmable gate arrays*. 23–32.
- [67] Hanchen Ye, Cong Hao, Jianyi Cheng, Hyunmin Jeong, Jack Huang, Stephen Neuendorffer, and Deming Chen. 2022. Scalehls: A new scalable high-level synthesis framework on multi-level intermediate representation. In *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 741–755.
- [68] Xiaofan Zhang, Yuhong Li, Junhao Pan, and Deming Chen. 2022. Algorithm/Accelerator Co-Design and Co-Search for Edge AI. *IEEE Transactions on Circuits and Systems II: Express Briefs* 69, 7 (2022), 3064–3070.
- [69] Xiaofan Zhang, Hanchen Ye, Junsong Wang, Yonghua Lin, Jinjun Xiong, Wen-mei Hwu, and Deming Chen. 2020. DNNExplorer: a framework for modeling and exploring a novel paradigm of FPGA-based DNN accelerator. In *Proceedings of the 39th International Conference on Computer-Aided Design*. 1–9.